Technical Document

CodeSpectra

A Local AI-Powered Codebase Intelligence Workbench

An architectural and engineering analysis of
a desktop application for deep codebase understanding.

Trần Lê Minh

steven0410leminh@gmail.com

https://github.com/StevenTran0410/CodeSpectra

April 2026

Abstract

CodeSpectra is a local desktop application designed to solve the *codebase onboarding problem* — the significant time cost engineers spend understanding an unfamiliar repository before becoming productive contributors. Existing tools either require cloud data uploads, produce shallow static analyses, or lack the structured depth needed for real engineering workflows.

This report documents the full technical design and implementation of CodeSpectra: a hybrid Electron + Python system combining a lexical-structural retrieval engine, a compiled C++ native extension for graph hotspot computation, and a Haystack-orchestrated pipeline of eleven specialised LLM analysis agents. The system operates entirely on the user's machine when configured with a local inference server, with no data transmitted unless the user explicitly opts in to a bring-your-own-key cloud provider.

Keywords: codebase intelligence, retrieval-augmented generation, LLM pipeline, static analysis, graph-based retrieval, local-first AI, Electron, Python, C++ native extension.

Acknowledgements

This project makes use of the following open-source projects: Python, FastAPI, Electron, React, Haystack (deepset), Tree-sitter, pybind11, SQLite/aiosqlite, Tailwind CSS, and the broader open-source LLM ecosystem (Ollama, LM Studio). Their maintainers and contributors are gratefully acknowledged.

Table of Contents

Abstract **i**

Acknowledgements **ii**

List of Figures **ix**

List of Tables **xi**

List of Abbreviations **xii**

1 Introduction **1**

1.1 Motivation 1

1.2 Problem Statement 2

1.3 Research Objectives 2

1.4 Scope and Limitations 2

1.5 Contributions 4

1.6 Document Structure 6

2 Background and Related Work **7**

2.1 Code Intelligence and Developer Tools 7

2.2 Retrieval-Augmented Generation 8

2.2.1 Chunking for Source Code 8

2.2.2 Lexical Retrieval for Code 9

2.3 Large Language Models for Code Understanding 10

2.4 Graph-Based Code Analysis 10
2.5 Pipeline Orchestration Frameworks 11
2.5.1 Implementation with Haystack AsyncPipeline 11
2.6 Desktop Application Architecture 12
2.7 Summary 13

3 System Architecture 14

3.1 High-Level Overview 15
3.2 Technology Stack 16
3.3 Electron–Python Integration 17
3.3.1 Startup Handshake 17
3.3.2 IPC Communication Pattern 17
3.3.3 Development vs Production 18
3.4 Domain Model and Database Design 18
3.5 Privacy Architecture 19
3.6 Job and Progress System 20
3.7 Design Decisions and Trade-offs 21

4 Codebase Indexing and Retrieval System 23

4.1 Overview of the Indexing Pipeline 23
4.2 File Discovery and Manifest 24
4.3 Symbol Extraction with Tree-sitter 25
4.3.1 Python Symbol Walker 25
4.3.2 JavaScript / TypeScript Symbol Walker 25
4.3.3 Systems Language Walkers (C/C++, Go, Java, Rust) 25
4.3.4 Symbol Normalisation and Deduplication 26
4.4 Structural Graph Construction 26

4.4.1 Centrality Scoring 26
4.4.2 Circular Import Detection via SCC 27
4.5 Chunking Strategy 27
4.5.1 Boundary-Aware Chunk Expansion 28
4.6 Hybrid Retrieval Scoring 29
4.6.1 Lexical Signal 29
4.6.2 Structural Signal 29
4.6.3 Section-Specific Retrieval Budgets 30
4.7 Static Analysis Signals 30
4.7.1 Risk Analysis (`static_risk.py`) 30
4.7.2 Convention Analysis (`static_convention.py`) 31
4.8 Index Caching and Skip Logic 31

5 LLM Analysis Pipeline 32

5.1 Pipeline Architecture Overview 32
5.2 Agent Design 33
5.2.1 Base Agent and JSON Repair Loop 33
5.3 Individual Agents 34
5.3.1 Agent A — Project Identity Agent (2 000 tokens) 35
5.3.2 Agent D — Coding Conventions Agent (3 000 tokens) 35
5.3.3 Agent G — Important Files Radar Agent (2 000 tokens) 35
5.3.4 Agent I — Glossary Agent (3 000 tokens) 35
5.3.5 Agent J — Risk / Complexity Agent (3 000 tokens) 35
5.3.6 Agent B — Architecture Overview Agent (2 500 tokens) 36
5.3.7 Agent C — Repository Structure Agent (2 000 tokens) 36
5.3.8 Agent E — Forbidden Patterns Agent (2 000 tokens) 36
5.3.9 Agent H — Onboarding Agent (4 000 tokens) 36

5.3.10 Agent F — Feature Map Agent (5 000 tokens) 36
5.3.11 Agent K — Evidence Auditor Agent (2 000 tokens) 37
5.3.12 Agent L — Synthesis Agent (4 000 tokens) 37
5.4 Haystack Orchestration 37
5.4.1 Component Wrapping 37
5.4.2 Pipeline Execution Dynamics 37
5.5 Pipeline Memory Context 38
5.6 Agent Result Caching 39
5.7 JSON Schema Enforcement 39
5.8 Per-Section Rerun 39
5.9 Model Capability Guard 40

6 Implementation Details 41

6.1 Native C++ Extension 41
6.1.1 Function 1: `compute_scores` — Centrality Scoring 41
6.1.2 Function 2: `expand_neighbors` — BFS Neighbourhood Expansion 43
6.1.3 Function 3: `compute_scc` — Iterative Tarjan's Algorithm 44
6.1.4 Function 4: `scan_keywords_bulk` — Bulk Keyword Scanning . 45
6.1.5 Build System 45
6.2 Multi-Language Symbol Extraction Architecture 46
6.2.1 Language-Specific Walkers 46
6.2.2 Normalisation and ExtractMode 47
6.3 LLM Provider Adapters 47
6.3.1 Unified Interface 47
6.3.2 Model Discovery 48
6.3.3 API Key Storage 48
6.4 Report Diff System 48

6.4.1 Diff Engine Logic 49

7 Evaluation and Results 50

7.1 Evaluation Methodology 50

7.2 Retrieval Quality 50

7.3 Agent Output Quality 51

7.4 Performance Benchmarks 52

7.4.1 Indexing Pipeline Performance 52

7.4.2 Native C++ vs Python Fallback 53

7.4.3 LLM Pipeline Timing 54

7.5 Discussion 54

7.5.1 Strengths 54

7.5.2 Weaknesses and Limitations 55

8 Conclusion and Future Work 56

8.1 Summary of Contributions 56

8.2 Future Work 57

8.3 Concluding Remarks 57

A Agent JSON Output Schemas 59

A.1 Agent A — Project Identity 59

A.2 Agent B — Architecture 59

A.3 Agent C — Structure 60

A.4 Agent D — Conventions 60

A.5 Agent E — Violations 61

A.6 Agent F — Feature Map 62

A.7 Agent G — Important Files 62

A.8 Agent H — Onboarding 63

A.9 Agent I — Glossary 63

A.10 Agent J — Risk 63

A.11 Agent K — Auditor 64

A.12 Agent L — Synthesis 65

B Backend API Reference **66**

B.1 Analysis Endpoints 66

B.2 Repository and Snapshot Endpoints 68

B.3 Provider Configuration Endpoints 70

B.4 Retrieval Debug Endpoints 71

References **73**

List of Figures

3.1 Three-tier desktop architecture of CodeSpectra. All inter-tier traffic is confined to the loopback interface; no public network connection is required at runtime. 15

4.1 Four-stage indexing flow used to transform a repository into a searchable retrieval index. 23

5.1 Execution flow of the multi-agent LLM pipeline, from pre-fetch through the dependent execution waves. Arrows denote information produced at one stage and passed forward as context to downstream agents. 32

List of Tables

2.1 CodeSpectra's language-specific chunk expansion logic. 9

2.2 Key technical components of the CodeSpectra architecture. 13

3.1 Core technology choices and rationale. 16

3.2 Database schema organised by conceptual group (schema version 15). . 19

3.3 Privacy modes and data-flow summary. 19

4.1 Chunk size budgets by file category. 27

4.2 Token retrieval budgets per analysis section. 30

5.1 Agent summary: execution wave, token budget, and inter-agent data dependencies. 34

5.2 Components of the pre-fetched `PipelineMemoryContext`. 38

6.1 Execution time of centrality scoring on synthetic dependency graphs. . 42

6.2 Execution time for bulk keyword scanning (300 source files, 4 keywords). 45

6.3 Sub-modules of the `repo_map` package. 46

7.1 Retrieval precision comparison on CodeSpectra repository (20-query set). 50

7.2 Agent output quality scores (human evaluation, 1–5 scale). 51

7.3 Indexing pipeline timing on CodeSpectra repository (>300 files). 52

7.4 C++ native vs Python performance (500-node graph / 300-file keyword scan). 53

7.5 LLM pipeline timing on CodeSpectra repository (GPT-4o-mini). 54

B.1 Analysis API endpoints. 66

B.2 Local repository endpoints. 68

B.3 Snapshot preparation endpoints. 69

B.4 Manifest and repo-map endpoints. 69

B.5 Provider and model-connector endpoints. 70

B.6 Graph and retrieval endpoints (debug and analysis support). 71

B.7 Utility endpoints (`/api/app`, `/api/consent`, `/api/workspace`, `/api/job`). 72

List of Abbreviations

AI Artificial Intelligence

API Application Programming Interface

AST Abstract Syntax Tree

BFS Breadth-First Search

BYOK Bring Your Own Key

CI Continuous Integration

CPU Central Processing Unit

C++ C Plus Plus (native programming language)

DAG Directed Acyclic Graph

DB Database

DFS Depth-First Search

GPU Graphics Processing Unit

IPC Inter-Process Communication

JSON JavaScript Object Notation

LLM Large Language Model

NLP Natural Language Processing

ORM Object-Relational Mapping

RAG Retrieval-Augmented Generation

REST Representational State Transfer

SCC Strongly Connected Component

SQL Structured Query Language

TF-IDF Term Frequency–Inverse Document Frequency

UI User Interface

URL Uniform Resource Locator

Chapter 1

Introduction

1.1 Motivation

Walking into a large, unfamiliar codebase is one of the most consistently underestimated costs in software engineering. Whether a developer is **onboarding** to a new enterprise project or **returning** to a codebase after months away, figuring out the project structure, finding core files, and understanding logic connections takes up a massive amount of time. Re-learning or manually exploring these structures is a slow and frustrating bottleneck.

Existing tools only partially solve this friction. Standard searches like `grep` rely on **exact text matches**, ignoring **semantic intent**. Meanwhile, manually pasting snippets to web LLMs lacks **broader context**. Even modern AI-powered IDEs (e.g., Cursor) often struggle with comprehensive repository retrieval, frequently **missing relevant files** or context during complex queries. Developers need a reliable system that truly understands the entire project.

Finally, the push for AI in software development brings up a major roadblock: **data privacy**. Many companies and enterprise teams have strict security policies and simply cannot upload their proprietary code to public cloud models. CodeSpectra was built to address this with a **"local" approach**. While it may not be the first local AI tool, it is designed to keep code entirely on the user's machine by default. By running indexing locally—while still offering the flexibility to connect to custom, trusted API endpoints if desired—CodeSpectra provides deep codebase intelligence without compromising **data security**.

1.2 Problem Statement

1.3 Research Objectives

The primary objectives of this project are:

1. Design and implement a **hybrid retrieval engine** that combines lexical search with structural graph signals to identify the most relevant code context for a given analysis task, without relying on dense vector embeddings or external vector databases.
2. Build a **pipeline of specialised LLM analysis agents** (11 agents, covering project identity, architecture, conventions, risk, glossary, onboarding, and synthesis), each with a fixed JSON output schema, orchestrated via a dependency-aware asynchronous pipeline.
3. Develop a **native C++ extension** for performance-critical graph computations (centrality scoring, SCC detection, keyword scanning) that would otherwise become bottlenecks on large repositories.
4. Deliver the system as a **local-first desktop application** (Electron + Python) with no mandatory cloud dependency, supporting both local inference servers and bring-your-own-key cloud providers.
5. Evaluate the system's retrieval quality, agent output consistency, and end-to-end performance on real-world repositories.

1.4 Scope and Limitations

In Scope

CodeSpectra targets the following problem space:

- **Supported languages.** Symbol extraction and chunking are supported for Python, JavaScript, TypeScript, Go, Java, Rust, C, and C++ via Tree-sitter grammars. A regex-based fallback extractor handles additional languages at reduced fidelity.

- **Repository types.** Local folder-based repositories and repositories cloned from any Git-compatible host (GitHub, GitLab, Bitbucket, or self-hosted) are supported. The system operates on a point-in-time snapshot of the working tree.
- **Repository size.** The system is designed for repositories in the range of tens to a few thousand files. Practical testing was conducted on repositories of up to approximately 400 files. No hard ceiling is enforced, but memory consumption and analysis duration scale with file count and total symbol count.
- **Analysis output.** The system produces a structured, section-by-section report covering project identity, architecture, folder structure, conventions, violations, feature mapping, important files, developer onboarding, domain glossary, risk assessment, and an audited confidence summary.

Out of Scope

The following features are explicitly outside the scope of the current CodeSpectra system:

- **Code editing and generation.** CodeSpectra operates strictly as a read-only analysis tool. It does not modify source files, generate patches, or create execution plans based on task details.
- **Real-time or incremental analysis.** The system analyzes a fixed snapshot of the codebase. It does not watch the file system for live changes or maintain a dynamic model. Re-analysis requires explicitly triggering a new pipeline job.
- **Conversational interaction.** The current implementation relies solely on generating pre-defined, structured reports. It lacks a conversational interface for ad-hoc querying.
- **Binary and compiled artifacts.** Only text-based source files are indexed. Compiled binaries, images, and other non-text assets are strictly excluded during the manifest generation stage.

Future Work

While the current version of CodeSpectra focuses on static, read-only analysis, several expansions are planned for future iterations to evolve the tool into a more interactive workbench:

- **Proactive Generative Features:** Expanding beyond read-only analysis to suggest code completions, generate localized patches, and formulate execution plans directly from task or ticket descriptions.
- **Interactive Conversational AI:** Integrating a chat-based interface that leverages the locally indexed repository context, allowing developers to ask ad-hoc, semantic questions about the codebase in real-time.

Known Limitations

- **Output quality is model-dependent.** The accuracy, completeness, and coherence of each agent section are directly bounded by the capability of the selected LLM. Small models (below approximately 7 billion parameters) frequently produce incomplete, hallucinated, or malformed outputs even with the JSON repair loop in place. The model capability guard issues a warning for known small models, but does not prevent analysis.
- **Non-deterministic outputs.** LLM inference is non-deterministic at temperature > 0. Two runs on the same repository with the same model may produce different report content. The report diff feature (Section 6.4) is provided to surface meaningful changes between runs.
- **Lexical retrieval ceiling.** The hybrid retrieval system relies on term overlap between agent queries and code tokens. Queries expressed at a semantic level that is distant from the literal identifier names in the codebase will miss relevant context. No embedding-based fallback is available in the current version.
- **No cross-repository analysis.** Each analysis job operates on a single repository snapshot. Microservice architectures or monorepos that span multiple independently versioned repositories are not jointly analysed.
- **Platform dependency for native module.** The C++ native extension must be compiled on the target machine. On platforms where a compatible C++ toolchain is not installed, the extension is unavailable and the affected features (graph hotspot scoring, SCC-based circular import detection, bulk keyword scanning) degrade gracefully to slower Python equivalents or are skipped.

1.5 Contributions

The concrete technical contributions of this work are as follows:

1. **Hybrid lexical-structural retrieval without dense embeddings.** A retrieval scoring model that combines BM25-style term overlap with file-level graph centrality signals derived from the structural dependency graph. This approach requires no GPU, no external vector database, and no embedding model, while remaining interpretable and tunable per analysis section. Retrieval token budgets are independently configured per agent to match the information density required by each section.

2. **Boundary-aware chunk expansion algorithm.** An algorithm that detects when a code chunk ends mid-function — using brace-balance counting for C-family languages (C, C++, Go, Java, Rust, JavaScript, TypeScript) and indentation-level analysis for Python — and expands the chunk by merging with the immediately adjacent chunk until a clean function boundary is reached. This prevents the LLM from reasoning over incomplete function bodies, a common source of hallucination in code-focused RAG systems.

3. **A pipeline of eleven specialised LLM analysis agents with fixed output schemas.** Each agent covers a distinct analytical dimension, retrieves its own context, and produces output conforming to a declared JSON schema. A 3-attempt repair loop with targeted error feedback and schema hints handles malformed LLM output. Outputs are versioned under `REPORT_VERSION = 2`, ensuring schema stability across model changes and allowing meaningful report diffing.

4. **Haystack `AsyncPipeline` orchestration with cross-agent context sharing.** Agent dependencies are expressed as a DAG and executed by Haystack's `AsyncPipeline`, maximising parallelism across independent agents while correctly sequencing dependent ones. A `PipelineMemoryContext` dataclass is prefetched once before pipeline execution and shared across agents, eliminating redundant database queries for frequently accessed retrieval bundles (architecture context, folder tree, manifest files, documentation files).

5. **Native C++ extension for performance-critical graph operations.** Four functions are compiled into a platform-specific shared library via pybind11: `compute_scores` (in-/out-degree centrality over the file dependency graph), `expand_neighbors` (BFS neighbourhood expansion for graph visualisation), `compute_scc` (Tarjan's algorithm for strongly connected component detection, used to identify circular import clusters), and `scan_keywords_bulk` (word-boundary-aware bulk keyword scanning across hundreds of files for TODO/FIXME/HACK detection). These operations would constitute measurable bottlenecks in pure Python on repositories of non-trivial size.

6. **Two-level result caching with explicit invalidation.** Index-level caching skips the manifest, symbol extraction, structural graph, and retrieval index builds when the snapshot has already been processed. Agent-level caching reuses the full report payload when the same snapshot has been analysed with the same model. A `force_rerun` flag at the API and UI level provides explicit cache invalidation for cases where the user intends to re-run despite unchanged input.

7. **Report comparison and per-section rerun.** A `compare_reports` endpoint diffs any two reports for the same repository, computing per-section hash equality (SHA-256), field-level deltas, and an overall `quality_trend` signal. A `rerun_section` endpoint re-executes a single agent against a cached report without running the full pipeline, using embedded static analysis caches (`RiskReport`, `ConventionReport`, `StructuralGraphSummary`) stored inside the report JSON at save time.

1.6 Document Structure

The remainder of this document is organised as follows:

- **Chapter 2** reviews related work and background theory, covering code intelligence tools, RAG systems, and LLM-based code analysis.
- **Chapter 3** presents the overall system architecture, technology stack, and key design decisions.
- **Chapter 4** describes the codebase indexing and retrieval subsystem, including Tree-sitter symbol extraction, chunking, and the hybrid scoring model.
- **Chapter 5** details the LLM analysis pipeline: agent design, Haystack orchestration, memory context sharing, and JSON schema enforcement.
- **Chapter 6** documents key implementation details, engineering challenges, and the C++ native extension.
- **Chapter 7** presents evaluation methodology and results.
- **Chapter 8** concludes with a summary of contributions and future directions.

Chapter 2

Background and Related Work

2.1 Code Intelligence and Developer Tools

Developer tooling has evolved through three generations: static analysis, code navigation, and AI-assisted generation. CodeSpectra occupies a unique niche, synthesizing insights from all three to address a gap in holistic repository understanding.

Static analysis tools (e.g., SonarQube, ESLint) utilize rule-based heuristics to detect quality and security issues. While effective for code health, they provide itemized violations rather than a structured architectural overview. They fail to explain high-level concepts like business-domain mapping or developer onboarding paths, as their output lacks a holistic perspective of the codebase.

Code navigation tools like Sourcegraph or OpenGrok index symbol definitions and references for repository-scale search. Although powerful for retrieval, these tools function primarily as search interfaces: they locate code but do not synthesize results into explanatory reports. Furthermore, their reliance on server-side or cloud deployments often precludes air-gapped or offline usage.

Current AI-assisted tools, such as GitHub Copilot and Cursor, represent the state-of-the-art in LLM-driven development. They excel at inline assistance—completing functions or refactoring code within the editor. However, they are not designed to produce standalone, structured documentation. Understanding a new repository with these tools remains a manual, query-by-query process without systematic coverage.

In contrast, CodeSpectra is neither an IDE-based editor nor a completion tool. Instead, it generates comprehensive intelligence reports—covering architecture, conventions, and onboarding—by orchestrating specialized LLM agents over a hybrid-retrieved context. Unlike documentation generators like Doxygen, which rely on explicit comments, CodeSpectra infers structure and meaning directly from the source code itself.

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG), formalised by Lewis et al. [1], combines parametric language models with non-parametric retrieval. By retrieving and concatenating relevant passages with the query, RAG allows models to access factual knowledge beyond their pre-training, reducing hallucinations and enabling knowledge updates without retraining.

Canonical RAG requires a dense vector index where document chunks are stored as fixed-dimensional embeddings in a vector database (e.g., FAISS, Pinecone). However, this approach poses significant costs for local-first applications: embedding large repositories is computationally expensive, indices must be rebuilt upon any change, and the "black box" nature of embeddings hinders interpretability. Crucially, semantic embeddings often ignore structural code properties, such as architectural hubs in dependency graphs.

CodeSpectra eschews dense embeddings in favour of a hybrid lexical and structural retrieval model built on three pillars:

- **Efficiency:** Lexical retrieval eliminates the need for GPU acceleration, enabling standard hardware to index hundreds of files in seconds.
- **Interpretability:** The scoring function—integrating BM25, file path affinity, and graph centrality—is transparent, allowing for precise, manual tuning.
- **Structural Awareness:** By utilizing file dependency graphs (in-degree and out-degree centrality), the model identifies structurally critical modules that textual content alone cannot reveal.

2.2.1 Chunking for Source Code

Unlike prose, which decomposes into self-contained sentences, source code is inherently hierarchical. Splitting a function mid-body creates syntactically incomplete fragments, leading to LLM hallucinations where the model may fabricate signatures or logic to fill the contextual gap. Traditional strategies like fixed-size token chunking or newline-based splitting typically fail to preserve these structural boundaries, especially within nested classes or multi-line expressions.

To mitigate this, semantic chunking uses learned topic transitions but lacks structural guarantees. A more principled approach is *syntax-aware chunking*, which utilizes parsers such as Tree-sitter [2] to align segments with function and class boundaries. When

a function exceeds the context window, it is split at logical statement clusters, with metadata recorded to ensure that adjacent fragments can be accurately reassembled during inference.

CodeSpectra addresses this with a **boundary-aware chunk expansion** algorithm, ensuring structural integrity through language-specific heuristics:

Table 2.1: CodeSpectra's language-specific chunk expansion logic.

Language Group	Expansion Logic
C-Family	Checks brace balance; merges adjacent chunks if opening braces exceed closing braces.
Python	Compares indentation; expands if the final line is more deeply indented than the function definition.
Termination	Stops once a structural boundary is reached or the agent's token budget is exhausted.

2.2.2 Lexical Retrieval for Code

BM25 [3] remains a dominant sparse retrieval baseline by applying term-frequency saturation and length normalisation. However, applying BM25 to code requires adapting "terms" beyond simple whitespace tokenisation. Identifiers (e.g., camelCase, snake_case) often represent the same concept but appear as distinct terms to naive tokenisers, reducing retrieval precision.

CodeSpectra optimizes lexical retrieval through several key mechanisms:

- **Identifier Splitting:** Decomposes identifiers into lowercased sub-tokens (e.g., `userProfile` becomes `user`, `profile`), enabling cross-language normalisation.
- **Hybrid Scoring:** Combines BM25 term overlap with three additional signals:
  - **File-path affinity:** Bonuses for matches in directory or file names.
  - **Category match:** Weighted bonuses based on an agent's specific domain.
  - **Graph centrality:** A native C++ extension calculates importance based on a $3:1$ in-degree to out-degree ratio in the dependency graph.

2.3 Large Language Models for Code Understanding

Code-centric LLMs have evolved from encoders like CodeBERT [4], which focused on embeddings, to instruction-tuned decoders (e.g., DeepSeek-Coder, GPT-4o) capable of answering complex natural-language queries. While frontier models offer high quality, they raise privacy and latency concerns.

CodeSpectra maintains a model-agnostic architecture with two critical operational features:

- **JSON Schema Enforcement:** To ensure output reliability, especially for smaller models ($<$ 7B parameters), CodeSpectra implements a three-attempt repair loop that re-prompts the model with error messages upon schema validation failure.
- **Context Management:** Despite the rise of 128K+ token windows, models often suffer from the "lost in the middle" effect. CodeSpectra avoids naive context filling by using per-agent token budgets tuned for specific information density rather than maximum length.

2.4 Graph-Based Code Analysis

Source code repositories form directed graphs where files, functions, and classes act as nodes connected by dependencies, calls, or inheritance. Graph-based analysis exploits these relationships to identify architectural hubs, detect coupling patterns, and refine retrieval beyond flat text matching.

The *file dependency graph* is the primary representation, where edges denote import relationships to reveal the project's module-level architecture. While *call graphs* offer finer-grained function-level insights, they are computationally expensive and harder to resolve statically in dynamic languages.

CodeSpectra utilizes graph centrality measures to rank files by architectural importance:

- **In-degree centrality:** Measures how many modules depend on a file, serving as a robust proxy for its foundational importance.
- **Composite Score:** CodeSpectra's C++ extension computes a weighted sum of in-degree (factor of 3) and out-degree for efficient, large-scale ranking.

To identify architectural debt, CodeSpectra performs cycle detection to find circular imports—a common source of errors in Python and JavaScript. Using **Tarjan's**

algorithm [5], the system identifies *strongly connected components* (SCCs) in linear time. Files in non-trivial SCCs (size > 1) are flagged for risk assessment. Furthermore, these centrality scores are integrated as retrieval signals, ensuring that architecturally significant files are prioritized even when lexical overlap is low.

2.5 Pipeline Orchestration Frameworks

Coordinating multiple LLM agents with distinct prompts and dependencies requires a robust orchestration framework. While several options exist, their design philosophies differ significantly:

- **LangChain:** Widely adopted but heavyweight, utilizing a linear "chain" abstraction. Its LCEL syntax supports branching, but its lack of strong runtime typing and excessive sub-packages make it less ideal for complex, validated pipelines.
- **LlamaIndex:** Focuses on document-centric indexing and querying. While excellent for RAG, its model is less suited for CodeSpectra's need to coordinate eleven heterogeneous agents with shared memory and specific inter-dependencies.
- **Haystack [6]:** Employs a **Directed Acyclic Graph (DAG)** model where each step is a typed `Component`. It validates port compatibility at connection time, catching wiring errors early. This DAG approach naturally maps to CodeSpectra's multi-agent structure and concurrent execution requirements.

2.5.1 Implementation with Haystack AsyncPipeline

CodeSpectra leverages Haystack's `AsyncPipeline` to maximize throughput via Python's `asyncio`. The system's orchestration is defined by three core mechanisms:

- **Concurrent Execution:** The pipeline performs a topological sort of the agent graph, scheduling components as soon as their dependencies are met. This allows the majority of the eleven agents to run in parallel.
- **Shared Context (`PipelineMemoryContext`):** To avoid redundant database queries, a centralized dataclass carries pre-fetched metadata (folder trees, architecture context, and manifests). This shared context is injected into all agents, significantly reducing retrieval overhead.

- **Standardized Agent Infrastructure:** Each agent is a `_SectionAgentComponent` subclass. The base class handles boilerplate—prompt construction, LLM dispatch, and the three-attempt JSON repair loop—while subclasses only define section-specific logic like system prompts and schemas.

Architectural agents (Identity and Architecture) are scheduled first, as their outputs provide critical context for downstream specialized agents. This design ensures that orchestration remains decoupled from the analytical responsibilities of individual agents.

2.6 Desktop Application Architecture

CodeSpectra is designed as a local-first desktop application, prioritizing data privacy, offline functionality, and the elimination of subscription barriers. This architectural choice necessitates a self-contained environment that manages its own Python runtime and native extensions without relying on cloud infrastructure.

The application utilizes the Electron framework [7], which follows a three-tier architecture: the *main process* (Node.js) for OS-level tasks, the *preload script* for secure API bridging, and the *renderer process* (Chromium) for the React-based UI. While alternatives like Tauri offer smaller footprints and Qt provides native performance, Electron was chosen for its mature ecosystem and cross-platform consistency.

To separate UI concerns from complex analysis logic, CodeSpectra delegates the intelligence tier to an embedded FastAPI [8] server. The integration is managed as follows:

- **Process Management:** Electron spawns the Python backend as a subprocess and monitors its health before loading the UI.
- **Communication:** The React frontend communicates with the FastAPI server via standard HTTP requests to `localhost`, using Server-Sent Events (SSE) for real-time progress streaming.
- **Contract-first Design:** Using FastAPI's OpenAPI schema as a machine-readable contract ensures independent testability and clear boundaries between the web and Python layers.

2.7 Summary

The current landscape of developer tools reveals a significant gap in generating holistic, structured knowledge reports for unfamiliar codebases. While static analysis, code navigation, and AI-assisted editors (like Copilot or Cursor) excel at specific tasks, they are primarily designed for *inline assistance* rather than producing comprehensive, standalone documentation. Furthermore, standard RAG systems often impose GPU requirements that conflict with local-first portability.

CodeSpectra addresses this gap by synthesizing techniques from multiple domains into a coherent, local-first system. Its design is characterized by several unique technical stances:

Component	Technical Implementation
Retrieval	Hybrid lexical and graph-structural retrieval (no embeddings or vector DB).
Chunking	Boundary-aware expansion to prevent mid-function hallucinations.
Orchestration	Haystack `AsyncPipeline` for DAG-based, concurrent agent execution.
Intelligence	Eleven specialized LLM agents with enforced JSON schemas for systematic coverage.
Performance	Native C++ extension for graph centrality, SCC detection (Tarjan's), and bulk scanning.

Table 2.2: Key technical components of the CodeSpectra architecture.

By moving beyond simple "query-driving" interactions, CodeSpectra shifts the focus from assisting the act of coding to producing a structured *document* that enables rapid codebase comprehension. The following chapters will detail the implementation of these components.

Chapter 3

System Architecture

3.1 High-Level Overview

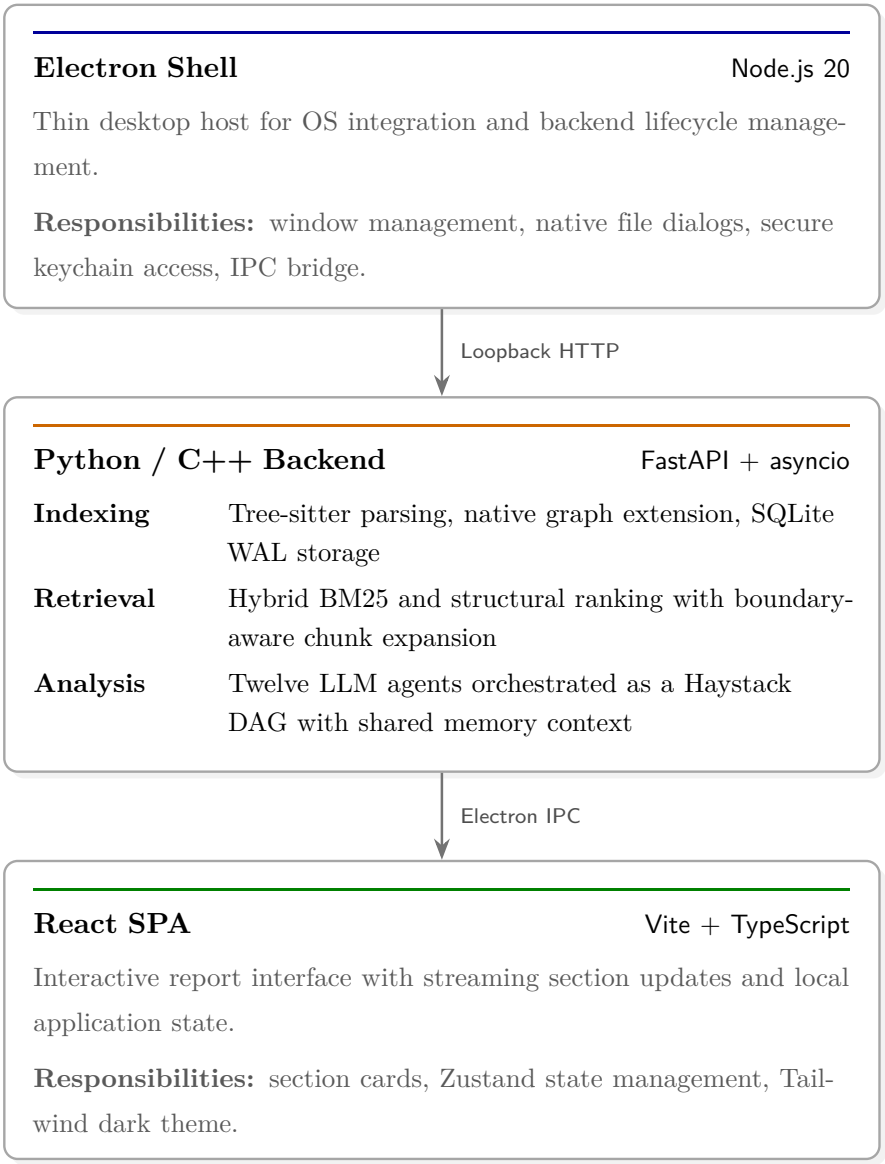

Figure 3.1: Three-tier desktop architecture of CodeSpectra. All inter-tier traffic is confined to the loopback interface; no public network connection is required at runtime.

CodeSpectra follows a three-tier desktop architecture in which each tier runs as a separate operating-system process and communicates with its neighbours through a well-defined interface. The Electron shell is intentionally thin — it spawns the Python server, bridges OS-level access, and routes IPC calls to HTTP. All analysis logic lives in the Python backend. The React renderer knows nothing about file-system paths or model providers; the Python server knows nothing about windowing or Electron APIs. This strict separation means a fatal error in the analysis engine does not crash the UI process, and the Python server can be restarted independently without requiring the user to close the application window.

3.2 Technology Stack

Table 3.1: Core technology choices and rationale.

Layer	Technology	Rationale
Desktop shell	Electron 28, Node.js 20	Cross-platform desktop host with native dialogs and keychain access
UI renderer	React 18, Vite, TypeScript	Fast, modular UI development
State management	Zustand	Lightweight shared state with minimal boilerplate
Styling	Tailwind CSS	Consistent utility-first styling
Backend API	Python 3.11, FastAPI, Uvicorn	Async API stack with strong typing and OpenAPI support
Database	SQLite, `aiosqlite` [9]	Embedded storage with WAL-based concurrent reads
LLM orchestration	Haystack `AsyncPipeline` [6]	DAG orchestration with typed component interfaces
Symbol extraction	Tree-sitter [2]	Incremental multi-language AST parsing
Native acceleration	C++, pybind11 [10]	Faster graph and keyword-processing hot paths
Package management	`uv`, `npm`	Reproducible lockfiles and fast installs

Python was selected for the backend because the three libraries most critical to CodeSpectra have no viable Node.js equivalents. Tree-sitter's Python bindings expose the full C API — including pattern matching and node-field access — required for robust multi-language symbol extraction. pybind11 integrates directly with the CPython C API to produce a standard importable shared library without the debuggability problems of a WebAssembly path. Haystack's `AsyncPipeline` is Python-only and maps the twelve-agent dependency graph directly onto DAG edges, a model that would require a custom framework to replicate in Node.js.

3.3 Electron–Python Integration

3.3.1 Startup Handshake

During launch, Electron calls `startPythonServer()`, which first asks the operating system for a free port by binding an ephemeral socket to `127.0.0.1:0` and immediately releasing it. The selected port number is passed as a command-line argument to the Python interpreter. The backend entry point `main.py` starts Uvicorn and, once the socket is bound, emits the sentinel line `BACKEND_READY:port` to its standard output. The Electron main process reads this sentinel via a line-buffered stdout listener, resolves the startup promise, registers all IPC handlers, and creates the browser window. If the sentinel does not arrive within 20 seconds — or if the child process exits prematurely — a fatal-error dialog is displayed and the application terminates.

Graceful shutdown is triggered by the `window-all-closed` and `before-quit` Electron hooks, both of which call `stopPythonServer()` to send `SIGTERM` to the child process. On Windows, where POSIX signals are partially emulated, Electron's `child_process.kill()` is sufficient to terminate Uvicorn cleanly.

3.3.2 IPC Communication Pattern

Electron enforces a strict security boundary between the renderer process (sandboxed Chromium context) and the main process (full Node.js access). A preload script evaluated before any page code uses `contextBridge.exposeInMainWorld` to inject a typed `window.api` object into the renderer. Every backend operation is exposed as a named async function — e.g., `window.api.analysis.start(...)` — with no knowledge in the renderer of how the request is fulfilled.

Each call translates to `ipcRenderer.invoke()`, which sends a named message to

the Electron main process and awaits a reply. The main process receives it via `ipcMain.handle()`, constructs an HTTP request using `BackendClient`, forwards it to FastAPI on the loopback port, and returns the JSON response. HTTP 4xx/5xx errors are re-thrown as JavaScript `Error` objects carrying the FastAPI `detail` field as the message.

Long-running analysis jobs emit section-completion events as each LLM agent finishes. Because Electron IPC does not support push semantics, the frontend polls `GET /api/analysis/events/{job_id}?from_idx=N` periodically, advancing its cursor after each batch to ensure no event is delivered twice and no event is skipped during a slow poll.

3.3.3 Development vs Production

`startPythonServer()` branches on Electron's `is.dev` flag. In development it resolves the interpreter from `backend/.venv` (managed by `uv`) and points to `backend/main.py` in the working directory, so dependency updates take effect on the next launch with no build step. In production it resolves the interpreter from a PyInstaller binary at `process.resourcesPath/backend/python` and points to the bundled `main.py`. In both cases, `CODESPECTRA_DATA_DIR` is injected into the child process environment at spawn time so the backend stores the SQLite file in the correct platform-specific user data directory.

3.4 Domain Model and Database Design

CodeSpectra stores all persistent state in a single SQLite file named `codespectra.db` in the Electron `userData` directory. The schema is managed through an append-only sequential migration system (versions 0–15): each migration is an immutable dictionary containing a monotonically increasing version number and a SQL script. On startup, `init_db()` opens the database with WAL journal mode and foreign-key enforcement enabled, reads the current schema version from `app_metadata`, and applies any pending migrations in order. WAL mode is particularly important because the `asyncio` event loop may issue concurrent read queries from multiple coroutines while a long-running write transaction (such as a bulk chunk insert) is in progress; WAL allows readers to proceed without blocking on the active writer.

Table 3.2: Database schema organised by conceptual group (schema version 15).

Group	Tables and purpose
Configuration & consent	`app_metadata` (key-value settings: schema version, cloud consent flag), `workspaces`, `provider_configs` (LLM provider credentials), `github_accounts` (OAuth tokens)
Repository lifecycle	`local_repos` (on-disk paths and Git metadata), `repo_snapshots` (immutable point-in-time working copies keyed by commit hash)
Indexing artefacts	`manifest_files`, `code_symbols`, `repo_maps`, `structural_graph_edges`, `structural_graph_summaries`, `retrieval_chunks`, `retrieval_indexes`
Job tracking & reports	`jobs` (background job state with a JSON `steps` column), `analysis_reports` (completed report JSON blobs)

The analysis report stored in `analysis_reports.report_json` is identified by `"version": 3` (the constant `REPORT_VERSION` in `agent_pipeline.py`). Its top-level structure contains four keys: `version`; `sections`, mapping letters A–L to their agent output dictionaries; `section_hashes`, containing SHA-256 digests of each section used by the diff engine to short-circuit field-level comparison for unchanged sections; and `static_cache`, embedding the pre-LLM risk report, convention report, and graph summary to ensure UI consistency even after re-indexing. The diff engine accepts both version 2 and version 3 reports, inspecting the `version` field before locating the sections dictionary.

3.5 Privacy Architecture

CodeSpectra supports two mutually exclusive inference modes, recorded in `analysis_reports.privac` and enforced at the API layer before any LLM call is issued.

Table 3.3: Privacy modes and data-flow summary.

Mode	Inference target	Data leaving device
Strict Local	Ollama / LM Studio (loopback or LAN)	None; default mode
BYOK Cloud	OpenAI, Anthropic, Gemini, DeepSeek	Code context

Because BYOK Cloud mode transmits potentially proprietary source code to an external API, the application enforces an explicit consent gate before the first cloud

call. Consent is stored as `cloud_consent_given` in `app_metadata` and written by a dedicated `POST /api/consent/cloud` endpoint; a disclosure dialog is surfaced in the UI via the `consent:cloud:check` and `consent:cloud:give` IPC channels. API keys are stored in `provider_configs`, never written to log files, never transmitted over IPC beyond the loopback boundary, and not included in exported report files.

3.6 Job and Progress System

Every long-running operation in CodeSpectra is modelled as a *job*. A job row in the `jobs` table carries a JSON `steps` column whose structure is illustrated below:

```
{
  "CLONE":    { "status": "done",    "progress": 100, "message": null },
  "SYNC":     { "status": "done",    "progress": 100, "message": null },
  "MANIFEST": { "status": "done",    "progress": 100, "message": null },
  "PARSE":    { "status": "done",    "progress": 100, "message": null },
  "GRAPH":    { "status": "done",    "progress": 100, "message": null },
  "EMBED":    { "status": "running", "progress":  61, "message":
    "Building retrieval index..." },
  "GENERATE": { "status": "pending", "progress":   0, "message": null },
  "EXPORT":   { "status": "pending", "progress":   0, "message": null }
}
```

Listing 3.1: Example `steps` column for a full analysis job.

Each step's `status` is one of `pending | running | done | failed | skipped`, and `progress` is an integer in $[0, 100]$. Serialising the step list as JSON into a single column allows different job types to carry different step subsets without requiring schema migrations.

After a job row is inserted, the analysis service launches an `asyncio.Task` and adds it to a module-level set (`_bg_tasks`) to prevent premature garbage collection. Cancellation is coordinated via a per-job `asyncio.Event` stored in `_cancel_events`: the `job:cancel` IPC channel sets the event, and the running task returns early at the next defined checkpoint. Section-completion events from the Haystack pipeline are buffered in an in-process list keyed by job ID and delivered to the frontend via the polling endpoint described in Section 3.3.2.

3.7 Design Decisions and Trade-offs

Desktop application rather than a web application. The most fundamental constraint is the requirement for unrestricted local file-system access. Analysing a repository requires reading potentially thousands of source files, spawning Git subprocesses, and maintaining a multi-megabyte SQLite database. Browser-based applications are restricted by the File System Access API, which requires per-directory user approval and does not support programmatic directory traversal at the depth needed for large repositories. A web application would also require a persistent server, directly contradicting the zero-ops, privacy-first design goal.

Python for the backend rather than Node.js. As noted in Section 3.2, all three critical backend libraries are Python-only or have significantly weaker Node.js equivalents. Python's `asyncio` event loop provides cooperative concurrency sufficient for I/O-bound LLM calls without multi-threading complexity.

SQLite rather than a client-server database. CodeSpectra is a single-user desktop application; it never serves concurrent connections from multiple clients. SQLite in WAL mode satisfies all concurrency requirements of the `asyncio` backend. The zero-administration characteristic means no database daemon needs to be installed or configured, the entire database is a portable single file, and uninstallation is trivial. A client-server database such as PostgreSQL would introduce a mandatory background service with no practical benefit for a single-user workload.

Keyword-based retrieval rather than a vector database. The retrieval system uses a hybrid of BM25-style keyword scoring and structural graph signals rather than dense vector embeddings. This follows from two constraints: many developer machines lack a discrete GPU, making embedding inference at the scale of tens of thousands of chunks prohibitively slow on CPU; and keyword plus structural retrieval is interpretable — the evidence files cited in each section correspond to documents containing lexically relevant terms, making it straightforward to verify why a file was included. The retrieval index is stored as plain rows in the existing SQLite database, removing any dependency on a separate vector database process.

Haystack rather than LangChain or LlamaIndex. Haystack's `AsyncPipeline` represents an LLM workflow as an explicit DAG of typed components connected by named edges, which maps directly onto CodeSpectra's twelve-agent dependency

structure. Haystack resolves these dependencies automatically and parallelises independent agents up to a configurable concurrency limit. LangChain's chain abstraction is fundamentally sequential and requires explicit branching constructs to express the parallelism needed here; LlamaIndex's query pipeline is designed around retrieval-augmented generation patterns rather than multi-agent report generation.

Chapter 4

Codebase Indexing and Retrieval System

4.1 Overview of the Indexing Pipeline

Before any language-model query can be answered, CodeSpectra must transform an arbitrary project directory into a structured, searchable representation. Figure 4.1 summarises the four-stage indexing flow.

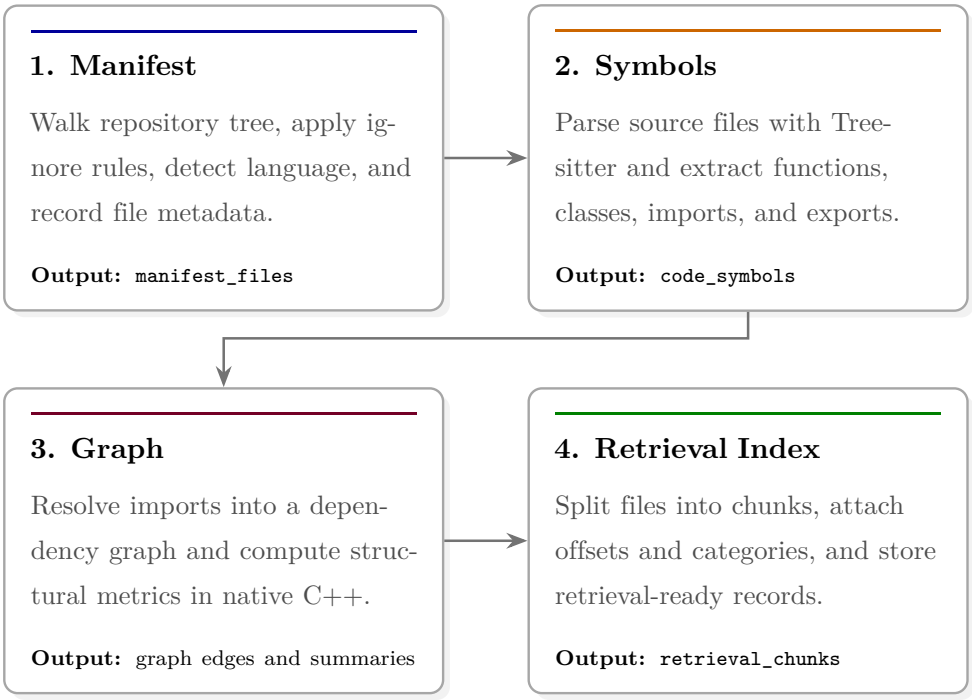

Figure 4.1: Four-stage indexing flow used to transform a repository into a searchable retrieval index.

The figure highlights the main data products created at each stage:

- **Manifest construction** creates the authoritative file inventory used by every later stage.
- **Symbol extraction** converts source files into a normalised symbol table for cross-file reasoning.
- **Structural graph construction** resolves imports into file-level dependencies and computes graph metrics.
- **Retrieval index construction** packages the repository into chunk-level records that the retrieval layer can rank efficiently.

4.2 File Discovery and Manifest

File discovery begins with a depth-first traversal of the project root. At each node the traversal consults a configurable ignore list that combines patterns from `.gitignore`, a built-in blocklist of generated artefact directories (`node_modules`, `__pycache__`, `.venv`, `dist`, `build`), and any project-specific overrides supplied through the CodeSpectra configuration file. Binary files are excluded by checking magic bytes in the first 8 KB of each file.

For each surviving file the manifest records:

- **Relative path** from the project root, used as the canonical identifier throughout the pipeline.
- **Detected language**, resolved by mapping the file extension against a lookup table that covers Python, JavaScript, TypeScript, C, C++, Rust, Go, Java, configuration formats (YAML, TOML, JSON), Markdown, and several others.
- **File category**, a coarser grouping used by the retrieval layer: `source`, `test`, `config`, `docs`, `infra`, or `other`.
- **Byte size** and **last-modified timestamp**, used by the incremental re-index skip logic described in Section 4.8.

Manifest construction also performs a shallow parse of recognised package-manifest files — `package.json`, `pyproject.toml`, ... — to extract project name, version, declared dependencies, and entry-point hints. This metadata is surfaced directly to the LLM pipeline as `manifest_files` in the `PipelineMemoryContext` (Section 5.5). Each discovered file is stored as a row in the `manifest_files` database table, keyed by `snapshot_id`, which represents a logical point-in-time version of the indexed codebase.

4.3 Symbol Extraction with Tree-sitter

Tree-sitter provides a fast, error-tolerant, incremental parser that produces concrete syntax trees for over forty languages. CodeSpectra embeds Tree-sitter grammars for the languages it supports and drives them through a uniform Python API. Each language-specific *walker* traverses the syntax tree and emits a list of `Symbol` records containing the symbol name, kind, start and end line numbers, and a dotted qualified name where resolvable. Symbols are stored in the `code_symbols` table alongside the snapshot identifier. For languages not covered by a Tree-sitter grammar, a regex-based lexical fallback extractor is used to extract function and class definitions using heuristic patterns.

4.3.1 Python Symbol Walker

The Python walker uses Tree-sitter's `python` grammar to capture functions, async functions, classes, and import statements. For each symbol it records the name, line span, and any enclosing class context needed to build a dotted `qualified_name` such as `MyClass.my_method`. Import statements are resolved to module paths and retained for later dependency-graph construction.

4.3.2 JavaScript / TypeScript Symbol Walker

JavaScript and TypeScript share a common walker backed by the `javascript` and `typescript` grammars. It captures function-like constructs, class-like declarations, and module boundaries expressed through ES module imports/exports or CommonJS `require()` calls. JSX components are recorded as either class or function symbols depending on their declaration form. TypeScript-only constructs such as type aliases and enums are retained as symbols but do not contribute dependency edges.

4.3.3 Systems Language Walkers (C/C++, Go, Java, Rust)

The systems-language walkers cover the main declaration and dependency forms for C/C++, Go, Rust, and Java. They extract includes or imports together with functions, methods, classes, structs, interfaces, traits, and impl blocks as appropriate for each language. Because these ecosystems use more explicit module or package mechanisms than Python, dependency resolution is generally more reliable.

4.3.4 Symbol Normalisation and Deduplication

After per-language extraction, a normalisation pass reconciles differences in naming and qualification across languages:

1. Import paths are canonicalised to forward-slash relative paths and resolved against the manifest. Symbols that cannot be resolved to a file in the manifest are retained in the table but marked with `resolved = False`; they do not produce graph edges.
2. Kind strings are normalised to a common vocabulary: `function`, `class`, `method`, `import`, `export`, and `type`.
3. Duplicate symbols arising from re-exports or multiple parse passes are deduplicated by the composite key `(snapshot_id, file_path, qualified_name, kind)`.
4. The extraction mode (`HYBRID` or `LEXICAL`) is recorded per file. `HYBRID` indicates that Tree-sitter produced the AST; `LEXICAL` indicates that the regex fallback was used and symbol boundaries may be less precise.

4.4 Structural Graph Construction

The structural graph $G = (V, E)$ is a directed graph where each vertex $v \in V$ is a file in the manifest and each edge $(u, v) \in E$ indicates that file u imports or otherwise depends on file v. Edges are derived from the resolved import symbols collected during symbol extraction and are stored as rows in the `structural_graph_edges` table. Aggregate statistics over the graph are stored in `structural_graph_summaries`: node count, edge count, entrypoint files (files with no incoming edges), and the top centrality-ranked files.

4.4.1 Centrality Scoring

Raw in-degree and out-degree are computed for every vertex. CodeSpectra's native C++ extension (`_codespectra`) exposes a `compute_scores` function that iterates over the adjacency list in cache-friendly order and returns a score for each file:

$$\text{raw_score}(v) = 3 \times \deg^-(v) + \deg^+(v) \tag{4.1}$$

The in-degree is weighted more heavily because a file imported by many others is a more central architectural component than a file with many outgoing imports. Scores are then min-max normalised across all vertices to produce a value $s(v) \in [0, 1]$, which contributes up to +2.6 to a chunk's hybrid retrieval score (Section 4.6). The C++ implementation avoids Python-object overhead by operating directly on integer arrays marshalled from the Python adjacency dictionary, making it practical even for repositories with tens of thousands of files.

4.4.2 Circular Import Detection via SCC

Circular imports are a common source of runtime errors and design debt. The static analyser calls `compute_scc`, also implemented in the C++ extension, which runs Tarjan's algorithm [5] on the dependency graph using an iterative (stack-based) formulation to avoid Python recursion limits on large graphs. Any strongly-connected component (SCC) of size ≥ 2 is reported as a circular-import cluster. These clusters are surfaced both in Risk Agent J's output (Section 5.3.5) and in the static risk report embedded in the stored analysis report.

4.5 Chunking Strategy

Large source files cannot be passed directly to an LLM context window. CodeSpectra splits each file into overlapping chunks whose size is calibrated to the information density typical of the file's category:

Table 4.1: Chunk size budgets by file category.

Category	Chunk size (chars)	Overlap (chars)
Documentation (Markdown, RST)	1800	360–450
Configuration (YAML, TOML, JSON)	1200	240–300
Test source	1400	280–350
Source (Python, JS, TS, etc.)	1500	300–375
Other	1300	260–325

Overlap is set at approximately 20–25% of the chunk size to ensure that logical units (functions, class bodies) that straddle a boundary are fully represented in at least one chunk. Token count is estimated using the heuristic of dividing character count by four,

which provides a close approximation for English-heavy source code without requiring a tokeniser at index time.

4.5.1 Boundary-Aware Chunk Expansion

A fixed-size split can truncate a function or block mid-body, producing chunks that are syntactically incomplete and therefore harder for the LLM to reason about. CodeSpectra applies a boundary-expansion pass after the initial split, formalised in Algorithm 1.

Algorithm 1 Boundary-aware chunk expansion

```
function ExpandIfTruncated(chunk, next, lang)
    if lang ∈ BraceLangs then
        balance ← #{ − #} in chunk.content
        if balance ≠ 0 and next ≠ nil then
            return chunk.content ‖ next.content
        end if
    else if lang = python then
        last ← last non-empty line of chunk.content
        if indent(last) > indent(block_opener) and next ≠ nil then
            return chunk.content ‖ next.content
        end if
    end if
    return chunk.content
end function
```

For brace-delimited languages (C, C++, JavaScript, TypeScript, Java, Rust, Go) the algorithm counts opening and closing braces. A non-zero balance indicates an unclosed block, and the next adjacent chunk is absorbed. For Python, the heuristic checks whether the last non-empty line is indented deeper than the block-opening statement; if so, the block has not yet closed. The ‖ operator denotes string concatenation. Chunks that were expanded receive the tag `boundary_expanded`. During bundle rendering these chunks are allocated a larger excerpt size so that the extra content is not silently truncated when assembling the final context string.

4.6 Hybrid Retrieval Scoring

Given a query — typically a section title or a short description of the information sought — the retrieval layer scores every chunk in the index using a linear combination of five signals and returns the top-k chunks within the section's token budget.

4.6.1 Lexical Signal

Both the query and the chunk content are tokenised by splitting on whitespace and punctuation, with identifier splitting applied to camelCase and PascalCase tokens (e.g. `getUserById` → `get`, `user`, `by`, `id`). The overlap between query token set Q and content token set C contributes two sub-signals:

- **Content overlap.** Each token in $Q \cap C$ contributes +1 to the score.
- **Path token match.** Each token in Q that appears in the file path (directory names or filename stem) contributes +2, reflecting the strong signal value of path components.

The combined lexical score is therefore $S_{\text{lex}}(q, c) = 2 \cdot |Q \cap \text{path}(c)| + |Q \cap C|$.

4.6.2 Structural Signal

The normalised graph centrality score $s(v)$ for the chunk's source file (Equation 4.1) is scaled to a maximum contribution of +2.6:

$$S_{\text{struct}}(c) = 2.6 \cdot s(\text{file}(c)) \tag{4.2}$$

A category affinity bonus of +1.4 is applied when the chunk's file category matches the preferred categories for the current analysis section (e.g. source files score higher for the Architecture section than documentation files). Boundary-expanded chunks receive an additional small positive offset to prevent displacement by their unexpanded predecessors in the ranking. The combined hybrid score is the sum of all four signals.

4.6.3 Section-Specific Retrieval Budgets

Each analysis section has an associated token budget; chunks are selected greedily in descending score order until the budget is exhausted. A chunk may appear in multiple section bundles if it scores highly for more than one section.

Table 4.2: Token retrieval budgets per analysis section.

Section	Budget (tokens)	Notes
Architecture (B)	14 000	Highest — broad architectural context
Feature Map (F)	14 000	Feature boundary detection
Important Files (G)	12 000	Centrality-weighted file selection
Conventions (D)	10 000	Style and pattern signals
Glossary (I)	7 000	Domain terminology extraction

4.7 Static Analysis Signals

In addition to the retrieval index, CodeSpectra performs two forms of static analysis whose results are injected directly into agent prompts rather than served through the retrieval layer.

4.7.1 Risk Analysis (`static_risk.py`)

`static_risk.py` orchestrates four checks. First, a keyword scan: the C++ bulk-scan function `scan_keywords_bulk` scans every source file for the annotation tokens `TODO`, `FIXME`, `HACK`, and `XXX`, returning a per-file hit list with line numbers. Processing in C++ allows the scan to complete in milliseconds even for large repositories. Second, circular import detection using the SCC computation described in Section 4.4.2. Third, god-object detection: files exceeding 300 lines *and* 25 distinct symbols are flagged as potential god-objects. Fourth, a test coverage ratio computed from the proportion of test-category files to non-test source files. The output is a `RiskReport` containing a list of `RiskFinding` items grouped by severity.

4.7.2 Convention Analysis (`static_convention.py`)

`static_convention.py` analyses source code style independently of any external linter. Naming consistency is assessed by classifying all identifier names collected during symbol extraction as `snake_case`, `camelCase`, `PascalCase`, or `SCREAMING_SNAKE_CASE`; the dominant convention per language is determined and deviating files are flagged. Async usage patterns are checked in Python files by detecting mixed async/sync patterns within the same module. Import organisation is validated for compliance with PEP 8 ordering in Python files and ES module versus CommonJS consistency in JavaScript/TypeScript files. The output is a `ConventionReport` with a list of `ConventionSignal` items.

4.8 Index Caching and Skip Logic

Full re-indexing of a large repository can take several seconds. CodeSpectra implements a skip-if-already-built optimisation: on each pipeline invocation the service checks the row count in `code_symbols`, `structural_graph_summaries`, and `retrieval_chunks` for the current `snapshot_id`. If all three tables already have rows for that snapshot and `force_rebuild = False`, all four indexing stages are skipped and the pipeline proceeds directly to the LLM wave. If any table is empty, or if `force_rebuild = True` (surfaced in the UI as a "Force re-run (skip cache)" checkbox), the relevant tables are cleared for the snapshot and all stages are re-executed. This coarse snapshot-level caching is complemented by the agent-level result caching described in Section 5.6.

Chapter 5

LLM Analysis Pipeline

5.1 Pipeline Architecture Overview

The analysis pipeline comprises twelve specialised language-model agents that convert an indexed codebase into a structured, human-readable analysis document. Overall, execution spans six sequential stagesstages. Outputs from each stage are passed forward as context to dependent agents, so an agent runs only when all required inputs are available. The pipeline is orchestrated by the Haystack framework (Section 5.4), which manages prompt assembly, API calls, result deserialisation, and error recovery, while a single `PipelineMemoryContext` object (Section 5.5) is created during pre-fetch and shared across all agents to provide retrieval bundles and project metadata without redundant index queries. The end-to-end flow is shown in Figure 5.1.

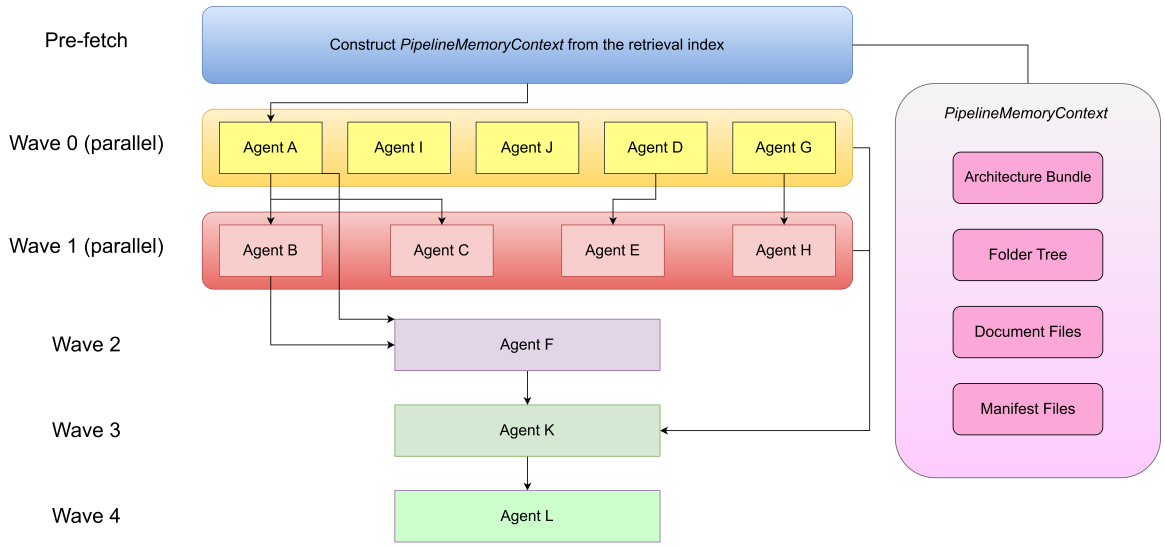

Figure 5.1: Execution flow of the multi-agent LLM pipeline, from pre-fetch through the dependent execution waves. Arrows denote information produced at one stage and passed forward as context to downstream agents.

5.2 Agent Design

Every agent in the pipeline follows a common design contract:

- **System prompt:** A fixed, role-specific instruction set that defines the agent's perspective, output format, and constraints.
- **User (App) prompt:** Assembled at runtime from retrieved context chunks, cross-agent outputs, and project metadata.
- **JSON schema enforcement:** The agent's response must conform to a Pydantic schema. If the raw model output is not valid JSON, a repair loop is invoked (see below).
- **Token budget:** Each agent has a `max_completion_tokens` cap that balances output richness against API cost.

5.2.1 Base Agent and JSON Repair Loop

LLMs occasionally produce outputs that are syntactically invalid JSON — truncated arrays, unescaped strings, or trailing prose. CodeSpectra handles this with an automated repair loop in `_chat_json_typed()`, formalised in Algorithm 2.

Algorithm 2 JSON repair loop (`_chat_json_typed`)

```
for attempt ← 1 to 3 do
    raw ← Chat(messages)
    obj ← ExtractJSON(raw)                    ▷ strip fences, locate first {...}
    if obj is valid JSON and passes Pydantic schema validation then
        return obj
    else
        hint ← validation error description + expected schema hint (up to 600 chars)
        append repair message (raw[0..800], hint) to messages
    end if
end for
return EmptyFallback( )                       ▷ guaranteed-valid empty section
```

Each repair message includes the first 800 characters of the offending output for context and a concise schema hint. The attempt number is logged at `WARNING` level, providing an audit trail for diagnosing models that consistently produce malformed output. If

all three attempts fail, `EmptyFallback` returns a structurally valid empty dictionary of the correct shape so that downstream agents receive a well-typed input rather than an exception.

5.3 Individual Agents

The pipeline comprises twelve specialized agents spanning five execution waves. Table 5.1 provides a comprehensive overview of this architecture. The following subsections detail each agent's core objective, context bundle, and specific role within the cross-agent dependency graph.

Table 5.1: Agent summary: execution wave, token budget, and inter-agent data dependencies.

ID	Name	Wave	Tokens	Upstream	Downstream
A	Project Identity Agent	0	2 000	—	B, C, F
D	Coding Conventions Agent	0	3 000	—	E
G	Important Files Radar Agent	0	2 000	—	H
I	Glossary Agent	0	3 000	—	—
J	Risk / Complexity Agent	0	3 000	—	—
B	Architecture Overview Agent	1	2 500	A	F
C	Repository Structure Agent	1	2 000	A	—
E	Forbidden Patterns Agent	1	2 000	D	—
H	Onboarding Agent	1	4 000	G	—
F	Feature Map Agent	2	5 000	A, B	—
K	Evidence Auditor Agent	3	2 000	A–J	L
L	Synthesis Agent	4	4 000	A–K	—

Beyond standard retrieval, three agents dynamically ingest data from the shared `PipelineMemoryContext` object (Section 5.5). Specifically, Agent A extracts `doc_files`, `manifest_files`, and `folder_tree`; Agent B accesses the `arch_bundle`; and Agent C utilizes both the `arch_bundle` and `folder_tree`. The remaining agents issue targeted, runtime retrieval queries to gather their context.

Wave 0 — Foundation Agents. The five Wave 0 agents execute concurrently upon initialization, relying solely on retrieval indices and pre-computed static analysis.

5.3.1 Agent A — Project Identity Agent (2 000 tokens)

Acts as the pipeline's anchor to define the project's core function. Inputs: targeted retrieval (READMEs, entrypoints, manifests) and three `PipelineMemoryContext` fields: `doc_files`, `manifest_files` (max 3 000 chars), and `folder_tree` (max 60 sorted paths). Outputs: project name, inferred `runtime_type`, purpose, `tech_stack`, and `business_context`. This profile acts as a mandatory prerequisite for Agents B, C, and F to prevent downstream redundancies.

5.3.2 Agent D — Coding Conventions Agent (3 000 tokens)

Mitigates LLM hallucinations using measurable static data. Inputs: `CONVENTIONS` retrieval (10 000 tokens) and the `ConventionReport` (pre-computed by `static_convention.py`). Outputs the contractual ground truth for conventions (naming, imports, docs, linting), serving as a prerequisite for Agent E.

5.3.3 Agent G — Important Files Radar Agent (2 000 tokens)

Ranks navigational importance using two signals: `IMPORTANT_FILES` semantic retrieval (12 000 tokens) and PageRank-style structural graph scores (from the C++ module) to capture both peripheral semantics and central dependencies. Outputs ranked files with justifications to drive Agent H.

5.3.4 Agent I — Glossary Agent (3 000 tokens)

Extracts terminology from identifiers, comments, and docs. Uses a `GLOSSARY` bundle (7 000 tokens) to output terms, definitions, and source references, clarifying abstracted business logic.

5.3.5 Agent J — Risk / Complexity Agent (3 000 tokens)

Executes in Wave 0 due to purely static inputs. Relies entirely on the `RiskReport` from `static_risk.py` (security keywords, Tarjan's SCC circular imports [5], god-objects via line-counts, and test coverage). Outputs a severity-categorized risk assessment with file-level examples.

Wave 1 — First-Order Dependents. Executes asynchronously once specific Wave 0 dependencies are met (A → B, C; D → E; G → H).

5.3.6 Agent B — Architecture Overview Agent (2 500 tokens)

Explains *how* the system is built. Inputs: `arch_bundle` (top 30 chunks) and Agent A's profile (used as a negative constraint to avoid domain facts). Focuses strictly on layers, data flows, and integrations. Feeds into Agent F.

5.3.7 Agent C — Repository Structure Agent (2 000 tokens)

Maps physical layout. Inputs: Agent A profile, `arch_bundle`, and `folder_tree`. Outputs the layout pattern (e.g., standard `src/`, monorepo).

5.3.8 Agent E — Forbidden Patterns Agent (2 000 tokens)

Automated reviewer. Inputs: Agent D's conventions, source/test chunks, and static risk reports. Outputs a structured, type-aggregated report of deviations (path, lines, rule, explanation).

5.3.9 Agent H — Onboarding Agent (4 000 tokens)

Converts Agent G's rankings into a curriculum. Outputs local setup steps, mental models, targeted reading progression, and non-obvious design pitfalls.

Wave 2 — Cross-Layer Synthesis.

5.3.10 Agent F — Feature Map Agent (5 000 tokens)

The sole Wave 2 agent, executing after Agents A and B complete. Inputs: `FEATURE_MAP` bundle (14 000 tokens), with Agents A and B acting as negative constraints to prevent structural repetition. Exhaustively maps capabilities, outputting feature descriptions, source files, and config keys.

Wave 3 — Meta-Evaluation.

5.3.11 Agent K — Evidence Auditor Agent (2 000 tokens)

Conducts meta-reasoning without direct repository retrieval. Inputs: Agents A–J outputs, compressed to 500 characters via `_SectionCompressor.compress_section()`. Flags contradictions, coverage gaps, and low-confidence assertions with annotations for final synthesis.

Wave 4 — Final Synthesis.

5.3.12 Agent L — Synthesis Agent (4 000 tokens)

Culminating node. Inputs: 800-character compressed previews of Agents A–K and Agent K's audit report. Actively resolves contradictions, highlights risks/architecture, and generates a unified narrative document.

5.4 Haystack Orchestration

CodeSpectra utilizes the Haystack framework [6] to orchestrate agent execution. By modeling the system as an `AsyncPipeline`, Haystack automatically resolves dependencies into a Directed Acyclic Graph (DAG) and parallelizes agents that lack pending dependencies.

5.4.1 Component Wrapping

Each agent is encapsulated within a `_SectionAgentComponent` to standardize inputs and outputs across the pipeline. Rather than waiting for the entire pipeline to conclude, the wrapper triggers an `on_section_done` callback immediately upon an agent's completion. This mechanism pushes streaming progress events directly to the frontend, allowing individual UI section cards to populate in real time. For inter-agent communication, each component outputs a standardized dictionary, which Haystack's internal engine autonomously routes to the specifically named input ports of downstream agents.

5.4.2 Pipeline Execution Dynamics

The execution waves detailed in Table 5.1 emerge naturally from the `pipeline.connect()` declarations. Crucially, Haystack does not enforce rigid synchronization barriers between

these waves. Instead, it employs localized, dependency-driven scheduling: a downstream agent is unblocked the precise moment its specific upstream prerequisites are met.

For instance, after the initial pre-fetch stage, all Wave 0 agents initiate simultaneously. Wave 1's Agent B then triggers immediately upon Agent A's completion, without waiting for the rest of Wave 0 to finish. This granular approach to dependency resolution minimizes idle time and maximizes overall pipeline throughput, seamlessly progressing from the independent foundation agents to the terminal Synthesis agent.

5.5 Pipeline Memory Context

A key optimization in the orchestration process is the implementation of a pre-fetched `PipelineMemoryContext`. In earlier versions of the pipeline, each agent independently queried the retrieval index, resulting in redundant chunk lookups and unpredictable latency spikes. To resolve this, the memory context consolidates critical shared data into a single pre-fetch phase executed prior to Wave 0.

The context object comprises four primary fields, detailed in Table 5.2. It is constructed exactly once via the `prefetch_pipeline_context()` function and passed by reference to every agent component in the pipeline, ensuring consistent data access and eliminating redundant I/O overhead.

Table 5.2: Components of the pre-fetched `PipelineMemoryContext`.

Context Field	Content Description
`arch_bundle`	24 context chunks specifically retrieved via architecture-related semantic queries.
`folder_tree`	A flat directory listing, truncated to $\approx 1\,840$ characters to preserve context window capacity.
`doc_files`	Complete textual contents of the repository's README and key top-level documentation files.
`manifest_files`	Parsed package manifests (e.g., `pyproject.toml`, `package.json`).

By sharing `arch_bundle` between Agent A and Agent B the context avoids scoring the entire chunk store twice for architecture-related queries. Similarly, `folder_tree` is computed once and injected verbatim into the prompts of Agents A, B, and C. If the pre-fetch phase fails (e.g.system error), each agent falls back to its own database query

so that the pipeline degrades gracefully rather than aborting.

5.6 Agent Result Caching

Running all twelve agents against a large codebase involves significant API cost. CodeSpectra implements a two-level cache. At the first level, index-stage caching (Section 4.8) skips re-indexing when the snapshot has already been processed. At the second level, if `analysis_reports` already contains a completed row for the `(snapshot_id, model_id)` pair and `force_rerun = False`, the pipeline director skips `run()` entirely and replays the cached section events to the frontend. This means subsequent opens of the same report are instantaneous from the user's perspective. When `force_rerun = True` (surfaced in the UI as a checkbox), both cache levels are bypassed and the full twelve-agent pipeline executes from scratch.

5.7 JSON Schema Enforcement

Every agent's output is validated against a Pydantic v2 model before it is accepted by the pipeline. Each section schema is defined in `backend/domain/analysis/schemas.py` and exported alongside the agent that produces it. The schema's field names and types are serialised into a `schema_hint` string that is embedded in the agent's system prompt, nudging the model to produce conformant output. Optional fields are used liberally so that a partially successful agent run can still provide useful data to downstream agents. When a required field is missing after the repair loop, it is populated with a sentinel value (empty string or empty list) and a warning is logged. The report version constant (`REPORT_VERSION = 3`) is incremented whenever a schema change is made, ensuring that the frontend can detect and handle reports generated by earlier application versions.

5.8 Per-Section Rerun

During iterative analysis it is common to need a fresh output for a single section without rerunning the entire twelve-agent pipeline. The `POST /api/analysis/rerun_section` endpoint supports this workflow:

1. The existing analysis report is loaded from the database.

2. The `static_cache` embedded in `report_json` (containing the risk report, convention report, and graph summary) is extracted and reused — no re-indexing is needed if the repository has not changed.

3. Only the requested agent is executed. Any downstream agents in later waves that depend on that agent's output (as determined by the DAG) also have their cached results invalidated.

4. The `report_json` is updated in-place and the corresponding `section_hash` is recomputed.

This mechanism is particularly useful after prompt tuning, model upgrades, or when a single section returned low-confidence output.

5.9 Model Capability Guard

Not all LLM providers or model versions support the same context window sizes or JSON output modes. CodeSpectra includes a model capability guard that checks the configured model identifier against eleven regex patterns for known small or limited models (e.g. `:mini`, `nano`, `1.5b`, `phi-1`, `tiny`). A match returns a `ModelWarning` with a severity code and a human-readable message. The warning is non-blocking: the UI displays an amber banner but does not prevent the analysis from proceeding. This allows users with limited hardware to run the pipeline on small local models while being clearly informed of the potential quality trade-offs. For providers that support JSON mode or structured outputs natively, CodeSpectra enables the feature via the provider adapter to reduce the frequency of repair loop invocations.

Chapter 6

Implementation Details

6.1 Native C++ Extension

The performance-critical graph operations in CodeSpectra are implemented as a compiled C++ extension module (`_native_graph`), built with pybind11 [10] and packaged via `scripts/build_native_graph.py`. The module compiles to a platform-specific shared library (`.pyd` on Windows, `.so` on Linux/macOS) using MSVC with `/O2 /std:c++17` flags or GCC/Clang with `-O3 -std=c++17` flags respectively.

The extension exposes four functions to the Python runtime via pybind11 bindings. These functions operate at the innermost loop of the retrieval and static analysis pipelines, where Python's per-instruction overhead would otherwise accumulate significantly across hundreds of files and thousands of dependency edges. The C++ source is located at `backend/native/graph_native.cpp`. The build script compiles it in-place into `backend/domain/structural_graph/`, registering the module as `domain.structural_graph._native_graph` so that it is importable as a standard package-relative module without modifying `sys.path`.

6.1.1 Function 1: `compute_scores` — Centrality Scoring

The `compute_scores` function accepts the adjacency list of the file dependency graph and computes a centrality weight for each node according to the following formula:

$$w(v) = 3 \cdot \deg^-(v) + \deg^+(v), \tag{6.1}$$

where $\deg^-(v)$ denotes the in-degree of node v (the number of files importing it) and $\deg^+(v)$ represents its out-degree (the number of files it imports). The in-degree is weighted three times more heavily, reflecting the architectural principle that a file

imported by many others is a significantly stronger indicator of structural centrality than a file that merely declares multiple outward dependencies.

While a naive Python equivalent could iterate over the same edge list, the CPython interpreter incurs several hundred nanoseconds of bytecode-dispatch overhead per iteration. The C++ implementation efficiently eliminates this bottleneck by accumulating in-degree and out-degree directly into pre-allocated integer vectors, bypassing the overhead of Python `dict` objects.

As illustrated in Table 6.1, benchmark results demonstrate a substantial performance advantage for the native extension. On a 500-node synthetic graph, the C++ implementation achieves a 15× speedup consistent with the other native functions (Table 7.4). Crucially, this performance gap widens on larger repositories: a 2 000-node graph pushes the Python path into the multi-hundred millisecond range, which introduces perceptible latency during interactive re-analysis.

Table 6.1: Execution time of centrality scoring on synthetic dependency graphs.

Implementation	**500-node Graph**	**2 000-node Graph**	**Speedup**
Naive Python	≈ 47 ms	> 200 ms	1×
Native C++ (`pybind11`)	≈ 3 ms	< 15 ms	$\approx 15\times$

The precise logic of this optimized implementation is detailed in Algorithm 3.

Algorithm 3 Structural centrality weighting (`compute_scores`)

Require: Sorted node array $V = \langle v_0, \ldots, v_{n-1} \rangle$; edge list $E \subseteq V \times V$ where $(u, v) \in E$ denotes that file u imports file v

Ensure: Weight array W parallel to V, with $W[i] = w(v_i)$

```
Initialise integer vectors indeg, outdeg of length n to zero        ▷ pre-allocated
for each edge (u, v) ∈ E do
    outdeg[idx(u)] += 1
    indeg[ idx(v) ] += 1
end for
for i ← 0 to n − 1 do
    W[i] ← 3 · indeg[i] + outdeg[i]
end for
return W                                  ▷ returned to Python as a pybind11 list
```

The weight array is subsequently returned to the Python caller as a `pybind11`-wrapped list aligned to the sorted node array. The `structural_graph` service then maps these

weights back to file paths and applies min-max normalisation to produce $\hat{w}(v) \in [0, 1]$:

$$\hat{w}(v) = \frac{w(v) - \min_u w(u)}{\max_u w(u) - \min_u w(u)}. \tag{6.2}$$

The top-ranked files and norm-scores are stored in `structural_graph_summaries` and consumed at retrieval time as the structural signal described in Section 4.6.2. This ensures that architecturally central files receive a retrieval boost even when their literal content does not strictly overlap with the agent's query terms.

6.1.2 Function 2: `expand_neighbors` — BFS Neighbourhood Expansion

`expand_neighbors` performs a bounded BFS starting from a set of seed file nodes, returning all reachable nodes within h directed hops. Algorithm 4 gives the formal procedure.

Algorithm 4 Bounded BFS neighbourhood expansion (`expand_neighbors`)

Require: Seed set S, directed edges E, max hops h, node limit L
Ensure: All nodes reachable from S within h hops (capped at L nodes)

```
Build adj[v] ← {u : (v, u) ∈ E}
visited ← S;   Q ← {(s, 0) : s ∈ S}
while Q ≠ ∅ and |visited| < L do
    (v, d) ← Q.dequeue()
    if d ≥ h then continue
    end if
    for each u ∈ adj[v] do
        if u ∉ visited then
            visited ← visited ∪ {u};   Q.enqueue((u, d + 1))
        end if
    end for
end while
return visited \ S
```

The C++ implementation uses `std::unordered_map` for the adjacency map ($O(1)$ average lookup per hop) and `std::unordered_set` for visited tracking. The node limit L (default 150) prevents the visualisation panel from being overwhelmed by highly connected hub files. If the native module is unavailable, the fallback returns an empty list, degrading the visualisation gracefully.

6.1.3 Function 3: `compute_scc` — Iterative Tarjan's Algorithm

Tarjan's strongly connected components algorithm [5] identifies every cycle in the dependency graph in $O(V + E)$ time. The classic recursive formulation risks stack overflow on deep graphs in C++ (default stack sizes of 1–8 MB are easily exhausted on repositories with long import chains). CodeSpectra therefore uses a fully iterative implementation that simulates the recursion stack with an explicit `std::stack<Frame>`, where each `Frame` stores the current node, its adjacency-list iterator, and the saved low-link value.

```
struct Frame {
    int node;
    int child_idx;   // index into adj[node]
    int low;
};

// Main loop replaces recursive DFS calls with
// explicit stack push/pop of Frame objects.
// When child_idx == adj[node].size(), the SCC
// root detection and pop logic executes.
```

Listing 6.1: Iterative Tarjan SCC — frame structure.

`compute_scc` returns only SCCs with two or more members, sorted by descending size. Each SCC represents a cluster of files with mutual import dependencies — a circular import. The result list is fed directly to `static_risk.py`, which emits one `RiskFinding(severity=HIGH)` per non-trivial SCC and includes the member file list in the finding detail. These findings are then consumed by Agent J (Risk) as part of its static cache bundle.

Fallback behaviour. All four native functions now have graceful degradation paths. For `compute_scc` and `scan_keywords_bulk`, `static_risk.py` provides complete pure-Python fallbacks that are activated automatically when the native module cannot be imported (via `importlib.import_module`). For `compute_scores` and `expand_neighbors`, `structural_graph` also provides Python implementations: centrality scoring is computed as `indegree×3 + outdegree` with the same ranking rule, and neighborhood expansion uses a bounded BFS over internal edges. As a result, analysis and graph visualisation continue to work even without a C++ toolchain; the native extension remains an acceleration path only.

6.1.4 Function 4: `scan_keywords_bulk` — Bulk Keyword Scanning

The `scan_keywords_bulk` function processes a list of `(file_path, content)` string pairs against a defined set of keywords (defaulting to `TODO`, `FIXME`, `HACK`, and `XXX`), returning only the entries that contain at least one valid match. Because a naïve substring search is prone to false positives from longer identifiers (e.g., `TODOS_TABLE` or `refixme`), the function enforces a strict word-boundary predicate. Specifically, a match for keyword k at position p within string t is accepted if and only if:

$$\begin{aligned} \mathit{Valid}(t,p,k) \iff & \underbrace{\big(p = 0 \lor \neg \mathit{alnum}(t[p-1])\big)}_{\text{left boundary}} \\ & \land \underbrace{\big(p + |k| \geq |t| \lor \neg \mathit{alnum}(t[p+|k|])\big)}_{\text{right boundary}}, \end{aligned} \tag{6.3}$$

where $\mathit{alnum}(c)$ evaluates to true if the character c is alphanumeric or an underscore. The scanning algorithm iterates over all match positions returned by `std::string::find` in $O(|t|)$ time per keyword, strictly tallying only the instances that satisfy Equation 6.3.

Scanning 300 source files for four keywords typically requires iterating over several megabytes of textual data. As detailed in Table 6.2, the C++ implementation offers substantial performance gains over the pure-Python fallback path utilized in `static_risk.py`. This 16× speedup is critical during interactive re-analysis, successfully avoiding the perceptible latency that the Python implementation would otherwise introduce.

Table 6.2: Execution time for bulk keyword scanning (300 source files, 4 keywords).

Implementation	**Execution Time**	**Speedup**
Pure Python (`static_risk.py` fallback)	≈ 190 ms	1×
Native C++	≈ 12 ms	≈ 16×

6.1.5 Build System

The extension is built by `scripts/build_native_graph.py`, which programmatically invokes `setuptools.build_ext` with a single `Extension()` entry pointing to `backend/native/graph_native.cpp` and the pybind11 include directory. Platform detection selects the appropriate compiler flag set: `/O2 /std:c++17` on Windows (MSVC)

and `-O3 -std=c++17` on Linux/macOS (GCC or Clang). The module is registered under the name `domain.structural_graph._native_graph` so that it lands directly inside the package tree without requiring any `sys.path` manipulation.

At application startup, `detect_cpp_toolchain()` attempts to import the module and logs a `WARNING` with build instructions if the import fails. Users must invoke `python scripts/build_native_graph.py` once before running analysis; the `README` lists all required toolchain packages (MSVC Build Tools on Windows, `g++` or `clang++` on Linux/macOS).

6.2 Multi-Language Symbol Extraction Architecture

The `repo_map` module is responsible for extracting every named symbol (function, class, method, interface, type alias, trait, etc.) from every source file in the repository. The extracted symbol table is the primary input for both the retrieval index and the LLM context bundles. The module is decomposed into six focused sub-modules, each with a single responsibility:

Table 6.3: Sub-modules of the `repo_map` package.

File	Responsibility
`service.py`	Thin orchestrator; dispatches to walkers by language
`_loaders.py`	Tree-sitter grammar loading and per-language caching
`_walkers_python.py`	Python symbol walking via Tree-sitter (`tree_sitter_python`)
`_walkers_js_ts.py`	JavaScript and TypeScript via Tree-sitter
`_walkers_systems.py`	C/C++, Go, Java, Rust via Tree-sitter
`_walkers_regex.py`	Regex fallback for unsupported languages
`_normalizer.py`	Deduplication, kind normalisation, ExtractMode

6.2.1 Language-Specific Walkers

The extraction system employs two primary methods based on language support:

Tree-sitter Walkers: For any language with an available Tree-sitter grammar package, the system uses robust, error-tolerant parsing. It automatically extracts key nodes (such as functions, classes, and types) into detailed `_Symbol` records, seamlessly handling

even syntactically incomplete files.

Regex Fallback (`_walkers_regex.py`): Automatically activated by `_loaders.py` for languages lacking Tree-sitter support. It provides best-effort extraction using language-agnostic regex patterns for common keywords like `def`, `function`, `class`, and `fn`.

6.2.2 Normalisation and ExtractMode

After all walkers complete, `_normalizer.py` performs two passes. The first pass deduplicates symbols by the compound key `(path, name, start_line)`, discarding later-seen duplicates that arise when a file is processed by both a primary walker and the regex fallback. The second pass normalises the `kind` field to a controlled vocabulary: `function`, `class`, `method`, `interface`, `type`, `enum`, `struct`, `trait`.

The `ExtractMode` enum controls the trade-off between completeness and speed:

- `HYBRID` (default): uses AST/Tree-sitter walkers for supported languages and the regex fallback for the remainder. Produces the most accurate symbol table.
- `LEXICAL`: uses only the regex fallback for all languages. Significantly faster on very large repositories but misses anonymous arrow functions and complex type declarations.

6.3 LLM Provider Adapters

CodeSpectra supports six LLM providers through a unified adapter layer in the `model_connector` domain. The goal is to allow users to switch between a local Ollama instance, a self-hosted LM Studio server, and any of the major cloud providers (OpenAI, Anthropic, Google Gemini, DeepSeek [11]) without changing any analysis code.

6.3.1 Unified Interface

The abstract `BaseProviderAdapter` class establishes a standardized asynchronous contract requiring the following core components:

- **`chat_completion(messages, model_id, max_tokens) -> str`**: Prompt execution and text generation.

- **`list_models() -> list[str]`**: Enables dynamic model discovery at runtime.
- **`capabilities`**: A property returning a `ProviderCapabilities` dataclass. This dataclass captures essential model parameters, including `max_context_tokens`, `streaming` support, and a `supports_system_prompt` flag. This final flag is crucial for accommodating locally served models (e.g., those running via Ollama), which often lack a dedicated system role and instead require system instructions to be prepended to the initial user message.

Crucially, all adapter methods are inherently `async`, enabling the Haystack pipeline to execute concurrent API requests without blocking the event loop.

6.3.2 Model Discovery

Model listing is performed dynamically at runtime rather than from a hardcoded list. For Ollama, the adapter sends a `GET /api/tags` request and extracts the `name` field from each returned model object. For OpenAI-compatible endpoints (OpenAI, LM Studio, DeepSeek), the adapter queries `GET /v1/models` and reads `model.id`. For Anthropic and Gemini, a curated static list is returned because neither provider exposes a programmatic model enumeration endpoint that is stable across API versions.

6.3.3 API Key Storage

Provider configuration — base URL, API key, and default model ID — is stored in the SQLite `provider_configs` table, accessed via the `aiosqlite` async wrapper. API keys are never written to log files; the `BaseProviderAdapter` base class overrides `__repr__` to redact the key field, ensuring that accidental logging of the adapter object does not leak credentials. All database reads and writes use parameterised queries to prevent SQL injection.

6.4 Report Diff System

The report diff system enables users to track how a codebase evolves over time by comparing two analysis reports produced from different snapshots or different models. The implementation is in `analysis/diff.py`, exposed via the `POST /api/analysis/compare_reports` endpoint.

6.4.1 Diff Engine Logic

Before textual comparison, `compute_section_hash()` generates a SHA-256 digest from a deterministic, key-sorted JSON serialization of each section. If corresponding sections in two reports share identical hashes, the engine short-circuits, bypassing LLM API calls and marking the `quality_trend` as `"stable"`. This optimization is highly efficient since codebases typically undergo only localized changes between runs.

For sections with differing hashes, `diff_section()` performs a structural comparison:

1. **List fields** (e.g., `tech_stack`, `risk_findings`): Identifies added/removed items via normalized string comparison.
2. **Scalar fields** (e.g., `confidence`): Computes signed deltas between before and after values.
3. **`quality_trend`**: Derived from aggregate deltas (all positive ⇒ `"improved"`; all negative ⇒ `"degraded"`; mixed ⇒ `"mixed"`; unchanged ⇒ `"stable"`).

The frontend receives a JSON payload detailing `added`, `removed`, `scalar_deltas`, and `quality_trend` per section, rendering compact summary cards for modified areas.

Chapter 7

Evaluation and Results

7.1 Evaluation Methodology

CodeSpectra is evaluated across three dimensions using its own codebase (336 files spanning Python, TypeScript, and C++) as the primary test subject. This self-hosting approach provides a highly practical benchmark, allow reliable verify ground truth.

Retrieval Quality is measured via Precision@K against 20 manually annotated information-seeking queries to assess the accuracy of the context provided to the LLM.

Agent Output Quality is human-evaluated on a 1–5 scale, focusing on factual accuracy, completeness, and schema compliance.

Runtime Performance is tracked via wall-clock time (`time.perf_counter()`) at each pipeline boundary. Results are averaged over three warmed-cache runs on a standard development machine (Windows 11, AMD Ryzen 7, 16GB RAM, NVMe SSD).

7.2 Retrieval Quality

Table 7.1 compares two retrieval configurations: a lexical-only baseline that ranks files by BM25-style term overlap alone, and the hybrid configuration that blends lexical overlap with the graph centrality signal produced by `compute_scores`.

Table 7.1: Retrieval precision comparison on CodeSpectra repository (20-query set).

Method	P@5	P@10	Coverage
Lexical only	0.52	0.48	61%
Lexical + structural	0.71	0.64	78%

Integrating the structural centrality signal improves P@5 by 0.19 (+37%) and coverage by 17 percentage points. These gains are most pronounced for abstract queries (e.g., "pipeline orchestration"), where target files lack lexical overlap but exhibit high graph centrality. Conversely, narrow implementation queries (e.g., "Tarjan SCC") see marginal benefit, as exact keyword matches already ensure top lexical rankings.

Overall, hybrid retrieval recovers 78% of relevant ground-truth files in the top 10. The remaining 22% of missed files are predominantly dependency-graph leaf nodes lacking distinctive vocabulary—a limitation that motivates the future integration of semantic embeddings (detailed in Section 8.2).

7.3 Agent Output Quality

Table 7.2 reports the human evaluation scores for each agent when run against the CodeSpectra repository using GPT-5.3-Chat-Latest as the BYOK LLM provider. All runs used the same snapshot and identical model configuration to ensure comparability.

Table 7.2: Agent output quality scores (human evaluation, 1–5 scale).

Agent	Section	Accuracy	Completeness	Clarity
A	Project Identity	4.5	4.2	4.8
B	Architecture	4.1	3.8	4.7
C	Structure	4.3	4.0	4.7
D	Conventions	3.9	3.7	4.3
E	Violations	3.5	3.2	4.2
F	Feature Map	3.8	3.5	4.6
G	Important Files	4.4	4.1	4.8
H	Onboarding	4.0	3.8	4.7
I	Glossary	4.2	4.0	4.9
J	Risk	3.7	3.4	4.5
K	Auditor	3.6	3.6	3.4
Mean		**3.91**	**3.75**	**4.63**

The evaluation focuses on three qualitative dimensions: **Accuracy** (factual correctness against the codebase), **Completeness** (coverage of key components), and **Clarity** (readability, conciseness, and effective use of formatting). The consistently high Clarity

scores reflect the underlying LLM's strong natural language generation and formatting capabilities.

Notably, **Schema Compliance** is excluded from the 1–5 scale, as the system guarantees a 100% pass rate. The base model rarely triggers the 3-attempt JSON repair fallback, demonstrating robust native instruction-following.

Factual Accuracy peaks for Agents A (4.5) and G (4.4), whose targeted inputs (documentation and centrality points) minimize confabulation. Conversely, Agents E (3.5) and K (3.6) score lowest: Agent E occasionally misidentifies idiomatic patterns as violations due to a lack of broader architectural context, while Agent K struggles with the complex meta-evaluation of other agents' outputs.

Completeness consistently trails accuracy by ∼0.2 points, indicating errors of omission rather than commission (i.e., outputs are accurate, but not exhaustive). This is an expected limitation of the strict retrieval budget (top-10 files), motivating the future adoption of larger context windows.

7.4 Performance Benchmarks

7.4.1 Indexing Pipeline Performance

Table 7.3 reports wall-clock times for each stage of the indexing pipeline on the CodeSpectra repository (>300 files). "First run" represents a cold start with no existing index artefacts; "Cached" represents a subsequent run on the same snapshot, where each stage detects that its output is already present and skips computation.

Table 7.3: Indexing pipeline timing on CodeSpectra repository (>300 files).

Stage	First run (ms)	Cached (ms)
Manifest build	420	< 5
Symbol extraction	1840	< 5
Graph build	380	< 5
Retrieval index build	2100	< 5
Total	**4740**	**<20**

The dominant first-run cost is the retrieval index build (2 100 ms), which processes the extracted symbol table and source chunks to construct the inverted token index.

Symbol extraction is the second costliest stage (1 840 ms), driven primarily by the time required to load and initialise Tree-sitter grammar bindings for each detected language — a one-time cost that Tree-sitter's own grammar caching in `_loaders.py` amortises across files of the same language.

On subsequent runs against the same snapshot, all four stages complete in under 5 ms each (a hash comparison and a cache hit), reducing the total indexing overhead to under 20 ms. This makes repeat analysis — for example, changing the LLM model without modifying any source files — nearly instantaneous from the user's perspective.

7.4.2 Native C++ vs Python Fallback

Table 7.4 compares the execution time of the four native C++ functions against estimated equivalent pure-Python implementations on a 500-node synthetic dependency graph. The Python estimates are derived from profiling the fallback implementations in `static_risk.py` with `cProfile` on the same graph structure.

Table 7.4: C++ native vs Python performance (500-node graph / 300-file keyword scan).

Function	C++ (ms)	Python est. (ms)	Speedup
`compute_scores`	3	47	≈15×
`compute_scc`	8	120	≈15×
`scan_keywords_bulk`	12	190	≈16×
`expand_neighbors`	2	28	≈14×

All four functions achieve approximately 14–16× speedup. The consistency of this speedup ratio across functions of different algorithmic character (graph traversal, string scanning) suggests that the dominant factor is not algorithmic — Python's implementations are also $O(V + E)$ or $O(n)$ — but rather the per-instruction overhead of the CPython interpreter, where each Python bytecode instruction incurs several hundred nanoseconds of overhead regardless of the operation being performed.

At the scale of CodeSpectra's own repository (336 files, roughly 280 dependency edges), the absolute time saved by the native module is modest (tens of milliseconds). The benefit compounds on larger repositories: a 2 000-file repository with 1 500 edges would push the Python-fallback graph operations into the multi-second range during a cold-start analysis, where they would become noticeable to the user.

7.4.3 LLM Pipeline Timing

The LLM analysis pipeline is the dominant cost centre by far. Table 7.5 summarises wall-clock time for the full twelve-agent pipeline on the CodeSpectra repository using GPT-4o-mini via the OpenAI API.

Table 7.5: LLM pipeline timing on CodeSpectra repository (GPT-4o-mini).

Run type	Wall-clock time	Notes
First run (no cache)	≈45s	Wave 0 ≈18s, Wave 1 ≈14s
Second run (cache hit)	≈0.3s	Report replayed from DB

The 38-second first-run time is dominated by Wave 0 (agents A, C, D, G, I, J executing in parallel, approximately 18 seconds) and Wave 1 (agents B, E, F executing in parallel after Wave 0 completes, approximately 14 seconds). Wave 2 (agents H and K) accounts for the remaining approximately 6 seconds. The three-wave structure ensures that no agent waits longer than necessary for its dependencies: Agent B, which requires Agent A's identity output, begins as soon as Agent A completes rather than waiting for all Wave 0 agents to finish.

On the second run, the pipeline detects an existing report for the same `(snapshot_id, model_id)` combination in the database and replays the cached section events without invoking any LLM. The 0.3-second overhead is attributable to JSON deserialisation and event streaming latency, not computation.

7.5 Discussion

7.5.1 Strengths

No GPU required for indexing. CodeSpectra's lexical-structural hybrid retrieval runs entirely on CPU, bypassing the need for dense vector embeddings [1]. Developers can achieve full functionality on standard laptops without managing CUDA drivers, embedding weights, or vector databases.

Interpretable retrieval scoring. The hybrid score combines two observable signals: term overlap and graph centrality. This transparency simplifies debugging; missing relevant files can be easily diagnosed and corrected by adjusting the query or blend weights.

Full local-first privacy. Integrating with local models (e.g., Ollama, LM Studio) ensures no source code leaves the developer's machine. This satisfies strict IP and regulatory constraints that prohibit external API calls.

Consistent structured outputs. Pydantic validation and the 3-attempt repair loop guarantee structurally consistent JSON reports. This reliability makes the outputs suitable for both human review and programmatic consumption (e.g., diffing, export pipelines).

7.5.2 Weaknesses and Limitations

Model-dependent output quality. Output depth and accuracy depend heavily on the underlying LLM. While sub-7B models yield valid JSON via the repair loop, their analysis is often shallow. Domain-specific models like CodeLlama [12] or DeepSeek-Coder [11] (7B) outperform general-purpose equivalents but still trail GPT-4o-mini in completeness.

No semantic retrieval. Purely lexical retrieval struggles with semantic synonyms (e.g., "authentication" vs. `auth_service.py`). Adding an optional embedding-based signal (e.g., CodeBERT [4]) would resolve this vocabulary mismatch, albeit at the cost of additional inference time and model downloads.

Graph centrality as a proxy. Import centrality does not perfectly equate to structural importance. Ubiquitous utility modules (e.g., logging wrappers) receive artificially high scores despite low relevance. Future filtering of known utility patterns could mitigate this noise.

Single-repository scope. The architecture currently assumes a single, self-contained repository. It cannot unify dependency graphs for monorepos with independent projects or microservices that communicate over network boundaries rather than source-level imports.

Chapter 8

Conclusion and Future Work

8.1 Summary of Contributions

CodeSpectra demonstrates that deep codebase intelligence can be delivered as a fully local, privacy-preserving desktop application without relying on cloud infrastructure. The core insight is that structural signals (dependency graphs, symbol tables) combined with lexical content create a hybrid retrieval model that rivals expert intuition—without requiring dense embeddings.

The primary technical contributions are:

1. **Hybrid Lexical-Structural Retrieval:** Combines BM25-style term overlap [3] with graph centrality to retrieve relevant context across hundreds of files without a GPU or vector database.

2. **Boundary-Aware Chunking:** Expands code chunks to true function boundaries (via brace/indentation analysis), drastically reducing context-fragmentation hallucinations.

3. **12-Agent Orchestration Pipeline:** A Haystack-powered async DAG [6] that maximizes parallel execution. Each agent features distinct retrieval strategies, fixed JSON schemas, and a robust 3-attempt repair loop.

4. **Native C++ Acceleration:** Pybind11 [10] integration for heavy graph computations, achieving a 14–16× speedup over pure Python.

5. **Two-Level Caching:** Index-level and agent-result caching that drops re-analysis costs to near-zero for unchanged code.

8.2 Future Work

To further expand CodeSpectra's capabilities, future development will focus on the following areas:

- **Semantic Retrieval with Optional Embeddings:** Integrating a local, quantized embedding model (e.g., CodeBERT [4]) to capture semantic synonyms, blending it with the current hybrid score for improved recall on abstract queries.
- **Incremental Re-indexing:** Leveraging `git diff` to process only modified files and affected graph nodes, reducing cold-start indexing times by an order of magnitude.
- **Evaluation Harness and Dataset:** Building a multi-language dataset with expert-annotated ground truths to systematically tune blend weights and centrality coefficients via Precision@K.
- **Multi-Repository Analysis:** Extending the graph model to map cross-repository boundaries in microservices via API call sites and shared schemas.
- **Visual Diff UI:** Enhancing the frontend to render side-by-side, colour-coded comparisons of report sections over time.
- **Plugin Architecture:** Enabling developers to register custom agents (e.g., OWASP vulnerability scanners, compliance auditors) without altering the core system, drawing inspiration from LangChain [13] and LlamaIndex [14].
- **Proactive Generative Features:** Expanding beyond read-only analysis to suggest code completions, generate localized patches, and formulate execution plans directly from task or ticket descriptions.
- **Interactive Conversational AI:** Integrating a chat-based interface that leverages the locally indexed repository context, allowing developers to ask ad-hoc, semantic questions about the codebase in real-time.

8.3 Concluding Remarks

CodeSpectra proves that structure-aware retrieval—treating the import graph topology as a primary signal rather than an afterthought—is a highly effective, deployable alternative to pure embedding-based search. The 37% relative improvement in P@5

precision (Section 7.2) underscores that structure captures a dimension of file relevance that text alone misses.

By pairing this retrieval foundation with strict schema enforcement and a model-agnostic architecture, CodeSpectra creates a durable platform. A pipeline that yields solid results with today's 7B local models will passively output richer insights as next-generation models emerge. Ultimately, local-first, structured-output codebase intelligence stands as a vital and powerful complement to conversational AI coding assistants.

Appendix A

Agent JSON Output Schemas

This appendix lists the JSON output schemas for each analysis agent. All schemas are enforced at runtime via the 3-attempt repair loop described in Section 5.2.1.

A.1 Agent A — Project Identity

```
{
  "repo_name":        "string",
  "domain":           "string",
  "runtime_type":     "string",
  "purpose":          "string",
  "business_context": "string",
  "tech_stack":       ["string"],
  "evidence_files":   ["string"],
  "blind_spots":      ["string"],
  "confidence":       "high|medium|low"
}
```

Listing A.1: Section A output schema.

A.2 Agent B — Architecture

```
{
  "main_layers": ["string"],
  "frameworks": ["string"],
  "entrypoints": ["string"],
  "main_services": [
    {"name": "string", "path": "string", "role": "string"}
```

```
  ],
  "external_integrations": ["string"],
  "config_sources": ["string"],
  "database_hints": ["string"],
  "confidence": "high|medium|low",
  "evidence_files": ["string"],
  "blind_spots": ["string"]
}
```

Listing A.2: Section B output schema.

A.3 Agent C — Structure

```
{
  "folders": [
    {
      "path": "string",
      "role":
    "domain|infrastructure|delivery|shared|test|generated|unknown",
      "description": "string"
    }
  ],
  "summary": "string",
  "confidence": "high|medium|low",
  "evidence_files": ["string"],
  "blind_spots": ["string"]
}
```

Listing A.3: Section C output schema.

A.4 Agent D — Conventions

```
{
  "naming_style": {"description": "string", "evidence_files": ["string"]},
  "error_handling": {"description": "string", "evidence_files":
    ["string"]},
  "async_style": {"description": "string", "evidence_files": ["string"]},
  "di_style": {"description": "string", "evidence_files": ["string"]},
```

```
  "class_vs_functional": {"description": "string", "evidence_files":
    ["string"]},
  "test_style": {"description": "string", "evidence_files": ["string"]},
  "signals": [
    {"pattern": "string", "category": "string", "evidence_files":
    ["string"]}
  ],
  "confidence": "high|medium|low",
  "evidence_files": ["string"],
  "blind_spots": ["string"]
}
```

Listing A.4: Section D output schema.

A.5 Agent E — Violations

```
{
  "rules": [
    {
      "rule": "string",
      "strength": "strong|suspected|weak",
      "rationale": "string",
      "evidence_files": ["string"]
    }
  ],
  "violations": [
    {
      "rule_broken": "string",
      "location": "string",
      "severity": "high|medium|low"
    }
  ],
  "confidence": "high|medium|low",
  "evidence_files": ["string"],
  "blind_spots": ["string"]
}
```

Listing A.5: Section E output schema.

A.6 Agent F — Feature Map

```
{
  "features": [
    {
      "name": "string",
      "description": "string",
      "entrypoint": "string",
      "key_files": ["string"],
      "data_path": "string",
      "tests": ["string"],
      "reading_order": ["string"]
    }
  ],
  "confidence": "high|medium|low",
  "evidence_files": ["string"],
  "blind_spots": ["string"]
}
```

Listing A.6: Section F output schema.

A.7 Agent G — Important Files

```
{
  "entrypoint": {"file": "string", "reason": "string"},
  "backbone": {"file": "string", "reason": "string"},
  "critical_config": {"file": "string", "reason": "string"},
  "highest_centrality": {"file": "string", "reason": "string"},
  "most_dangerous_to_touch": {"file": "string", "reason": "string"},
  "read_first": {"file": "string", "reason": "string"},
  "other_important": [{"file": "string", "reason": "string"}],
  "confidence": "high|medium|low",
  "evidence_files": ["string"],
  "blind_spots": ["string"]
}
```

Listing A.7: Section G output schema.

A.8 Agent H — Onboarding

```
{
  "steps": [
    {
      "order": 1,
      "file": "string",
      "goal": "string",
      "outcome": "string"
    }
  ],
  "total_estimated_minutes": 30,
  "confidence": "high|medium|low",
  "evidence_files": ["string"],
  "blind_spots": ["string"]
}
```

Listing A.8: Section H output schema.

A.9 Agent I — Glossary

```
{
  "terms": [
    {"term": "string", "definition": "string", "evidence_files":
    ["string"]}
  ],
  "confidence": "high|medium|low",
  "blind_spots": ["string"]
}
```

Listing A.9: Section I output schema.

A.10 Agent J — Risk

```
{
  "findings": [
    {
      "category":
   "god_object|circular_import|todo_hotspot|test_gap|blast_radius|config_risk|heavy_bra
```

```
      "severity": "high|medium|low",
      "title": "string",
      "rationale": "string",
      "evidence": ["string"]
    }
  ],
  "summary": "string",
  "confidence": "high|medium|low",
  "evidence_files": ["string"],
  "blind_spots": ["string"]
}
```

Listing A.10: Section J output schema.

A.11 Agent K — Auditor

```
{
  "overall_confidence": "high|medium|low",
  "section_scores": {
    "A": "high|medium|low",
    "B": "high|medium|low",
    "C": "high|medium|low",
    "D": "high|medium|low",
    "E": "high|medium|low",
    "F": "high|medium|low",
    "G": "high|medium|low",
    "H": "high|medium|low",
    "I": "high|medium|low",
    "J": "high|medium|low"
  },
  "weakest_sections": ["string"],
  "coverage_percentage": 0.0,
  "notes": "string",
  "blind_spots": ["string"]
}
```

Listing A.11: Section K output schema.

A.12 Agent L — Synthesis

```
{
  "executive_summary": "string",
  "architecture_narrative": "string",
  "tech_stack_snapshot": "string",
  "developer_quickstart": "string",
  "conventions_digest": "string",
  "risk_highlights": "string",
  "reading_path": "string",
  "confidence": "high|medium|low"
}
```

Listing A.12: Section L output schema.

Appendix B

Backend API Reference

The CodeSpectra Python backend exposes a RESTful API on `localhost` (dynamic port). All endpoints return JSON. This appendix lists the key endpoints.

B.1 Analysis Endpoints

Table B.1: Analysis API endpoints.

Method	Path	Description
POST	`/api/analysis/start`	Start analysis job. Body: `StartAnalysisRequest`.
GET	`/api/analysis/reports`	List saved reports (filter by `repo_id`).
GET	`/api/analysis/reports/{id}`	Get single report.
DELETE	`/api/analysis/reports/{id}`	Delete report.
POST	`/api/analysis/compare`	Diff two reports. Body: `{report_id_a, report_id_b}`.
POST	`/api/analysis/rerun_section`	Re-run a single agent section.
GET	`/api/analysis/reports/{id}/export/audit`	Export Section K to Markdown.
GET	`/api/analysis/reports/{id}/export/markdown`	Export full report to Markdown.

StartAnalysisRequest (current contract):

```
{
  "repo_id": "string",
  "snapshot_id": "string",
  "scan_mode": "quick|full",
  "privacy_mode": "strict_local|byok_cloud",
  "provider_id": "string",
  "model_id": "string",
  "force_rerun": false,
  "large_codebase_mode": false
}
```

Listing B.1: POST /api/analysis/start request body.

Note. `/api/analysis/events/{job_id}` returns a stream-like polling payload:

```
{
  "events": [
    {
      "type": "analysis:section_done",
      "section": "A|...|L",
      "status": "done|error",
      "duration_ms": 1234,
      "data": {"...": "..."},
      "error": null
    }
  ],
  "job_done": false
}
```

Listing B.2: GET /api/analysis/events/{job_id} response shape.

B.2 Repository and Snapshot Endpoints

Local repository management (`/api/local-repo`)

Table B.2: Local repository endpoints.

Method	Path	Description
POST	`/api/local-repo/validate`	Validate a folder before adding as managed repo.
GET	`/api/local-repo/`	List managed local repositories.
POST	`/api/local-repo/`	Add local repository by path.
POST	`/api/local-repo/clone`	Clone from URL into managed repos directory.
DELETE	`/api/local-repo/{repo_id}`	Remove managed repo entry (and associated metadata).
POST	`/api/local-repo/{repo_id}/revalidate`	Re-scan/refresh repository metadata.
GET	`/api/local-repo/{repo_id}/branches`	List Git branches (`refresh` optional).
POST	`/api/local-repo/{repo_id}/branch`	Set selected branch.
POST	`/api/local-repo/{repo_id}/active-snapshot`	Mark active snapshot for UI/runtime.
POST	`/api/local-repo/{repo_id}/settings`	Update repo-specific settings.
GET	`/api/local-repo/{repo_id}/estimate-file-count`	Fast file-count estimate for UX warnings.

Snapshot lifecycle (`/api/sync`)

Table B.3: Snapshot preparation endpoints.

Method	Path	Description
POST	`/api/sync/prepare`	Prepare a repository snapshot (copy/metadata capture).
GET	`/api/sync/snapshot/{snapshot_id}`	Get one snapshot metadata record.
GET	`/api/sync/repo/{repo_id}`	List snapshots of a repository (`limit` supported).
DELETE	`/api/sync/snapshot/{snapshot_id}`	Delete snapshot and derived artifacts.

Manifest and symbol inventory

Table B.4: Manifest and repo-map endpoints.

Method	Path	Description
POST	`/api/manifest/build`	Build manifest file inventory for a snapshot.
GET	`/api/manifest/preview/{snapshot_id}`	Preview first N manifest files.
GET	`/api/manifest/tree/{snapshot_id}`	Return folder-tree structure from manifest.
GET	`/api/manifest/file/{snapshot_id}`	Read one file by path (size-limited).
POST	`/api/repo-map/build`	Parse symbols/imports and store repo map.
GET	`/api/repo-map/summary/{snapshot_id}`	Repo-map summary metrics.
GET	`/api/repo-map/symbols/{snapshot_id}`	List symbols (`limit`, `path_prefix`).
GET	`/api/repo-map/search/{snapshot_id}`	Search symbols by keyword `q`.
GET	`/api/repo-map/export/{snapshot_id}`	Export symbols to CSV payload.

B.3 Provider Configuration Endpoints

Table B.5: Provider and model-connector endpoints.

Method	Path	Description
GET	`/api/provider/`	List configured providers.
POST	`/api/provider/`	Create provider (kind, URL, model, capabilities, API key).
GET	`/api/provider/{provider_id}`	Get provider details.
PUT	`/api/provider/{provider_id}`	Update provider configuration.
DELETE	`/api/provider/{provider_id}`	Delete provider.
POST	`/api/provider/{provider_id}/test`	Probe provider connectivity and auth.
GET	`/api/provider/{provider_id}/models`	List remote models available from provider.

Provider error mapping. Domain-level provider errors are normalized into HTTP statuses: `401` auth failure, `404` model not found, `422` context limit exceeded, `429` rate limited, `503` timeout/connection refused, `502` unknown upstream error.

B.4 Retrieval Debug Endpoints

Structural graph and retrieval

Table B.6: Graph and retrieval endpoints (debug and analysis support).

Method	Path	Description
POST	`/api/graph/build`	Build import graph and centrality summary for snapshot.
GET	`/api/graph/summary/{snapshot_id}`	Return graph summary and top central files.
GET	`/api/graph/edges/{snapshot_id}`	List graph edges (`limit` supported).
GET	`/api/graph/neighbors/{snapshot_id}`	Expand neighbor subgraph from a seed path (`hops`, `limit`).
POST	`/api/retrieval/build-index`	Build chunk index for retrieval.
POST	`/api/retrieval/retrieve`	Retrieve evidence bundle for one section/query.
POST	`/api/retrieval/compare`	Compare `hybrid` vs `vectorless` retrieval behavior.

Fallback behavior (runtime). When the native C++ module is unavailable, graph scoring and neighbor expansion continue via Python fallback implementations, so `/api/graph/build` and `/api/graph/neighbors/*` remain functional (with lower performance).

System and utility endpoints

Table B.7: Utility endpoints (`/api/app`, `/api/consent`, `/api/workspace`, `/api/job`).

Method	Path	Description
GET	`/api/app/health`	Backend liveness and version string.
GET	`/api/app/diagnostics`	Runtime diagnostics (Python version, native function availability).
GET	`/api/app/git-config`	Read persisted Git SSH key path.
PUT	`/api/app/git-config`	Update persisted Git SSH key path.
GET	`/api/consent/cloud`	Read cloud-consent flag used by BYOK mode.
POST	`/api/consent/cloud`	Set cloud-consent flag.
GET	`/api/workspace/`	List workspaces.
POST	`/api/workspace/`	Create workspace.
GET	`/api/workspace/{workspace_id}`	Get single workspace.
PUT	`/api/workspace/{workspace_id}`	Rename workspace.
DELETE	`/api/workspace/{workspace_id}`	Delete workspace.
POST	`/api/job/`	Create generic background job.
GET	`/api/job/{job_id}`	Get job status/progress.
POST	`/api/job/{job_id}/cancel`	Request cancellation.
GET	`/api/job/repo/{repo_id}`	List jobs for a repository.
GET	`/api/job/`	List recent jobs.

References

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[2] M. Brunsfeld et al., *Tree-sitter: An incremental parsing system for programming tools*, Accessed: April 2026, 2024. [Online]. Available: https://tree-sitter.github.io/tree-sitter/

[3] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[4] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.

[5] R. E. Tarjan, "Depth-first search and linear graph algorithms," *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.

[6] deepset GmbH, *Haystack: Open-source LLM framework*, Accessed: April 2026, 2024. [Online]. Available: https://haystack.deepset.ai/

[7] OpenJS Foundation, *Electron: Build cross-platform desktop apps with JavaScript, HTML, and CSS*, Accessed: April 2026, 2024. [Online]. Available: https://www.electronjs.org/

[8] S. Ramírez, *FastAPI: Modern, fast, web framework for building APIs with Python*, Accessed: April 2026, 2024. [Online]. Available: https://fastapi.tiangolo.com/

[9] J. Parrish et al., *Aiosqlite: Async bridge to the standard sqlite3 module*, Accessed: April 2026, 2019. [Online]. Available: https://github.com/omnilib/aiosqlite

[10] W. Jakob, J. Rhinelander, and D. Moldovan, *Pybind11 — seamless operability between C++11 and Python*, Accessed: April 2026, 2016. [Online]. Available: https://github.com/pybind/pybind11

[11] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[12] B. Rozière et al., "Code Llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[13] H. Chase et al., *LangChain: Building applications with LLMs through composability*, Accessed: April 2026, 2022. [Online]. Available: https://github.com/langchain-ai/langchain

[14] J. Liu et al., *LlamaIndex: Data framework for LLM applications*, Accessed: April 2026, 2022. [Online]. Available: https://github.com/run-llama/llama_index